



Basi di Dati

Progetto A.A. 2023/2024

Risiko online

0268142

Alessandro Massari

Indice

1. **Descrizione del Minimondo** **2**
2. **Analisi dei Requisiti** **4**
3. **Progettazione concettuale** **10**
4. **Progettazione logica** **19**
5. **Progettazione fisica** **26**

1. Descrizione del Minimondo

Sistema di gioco Risiko online

Si vuole realizzare un servizio online che consenta di giocare ad un clone del famoso gioco Risiko, in modalità "conquista del mondo".

Al sistema hanno accesso due tipologie di utenti: i giocatori e i moderatori. I moderatori hanno la possibilità di creare stanze di gioco, in funzione della quantità di giocatori che utilizzano attualmente il sistema. In particolare, i moderatori hanno la possibilità di visualizzare, tramite un report, quante stanze hanno attualmente partite in corso e quanti giocatori stanno partecipando alla partita. Inoltre, in questo report, gli amministratori possono visualizzare il numero totale di giocatori che hanno effettuato almeno un'operazione negli ultimi 15 minuti che non sono all'interno di alcuna stanza di gioco.

Una stanza permette ad un numero massimo di sei giocatori di entrare e partecipare alla partita. Una partita coinvolge almeno tre giocatori. Quando il terzo giocatore entra in una stanza, viene attivato un countdown tale da avviare la partita dopo due minuti. In questi due minuti altri giocatori possono entrare, fino al massimo concesso.

All'avvio della partita, gli stati del tabellone vengono assegnati casualmente ai giocatori. I turni "girano" in funzione del tempo di ingresso dei giocatori nella stanza (chi è entrato prima gioca prima). Un turno prevede che il giocatore possa compiere una delle seguenti azioni:

* Posizionare un numero arbitrario di carri armati in uno stato
* Scegliere uno stato da cui fare partire un attacco verso uno stato adiacente
* Spostare carri armati da uno stato ad un altro adiacente (almeno un carro armato deve restare nello stato di partenza)

L'attacco viene svolto nel seguente modo. La fase di attacco si svolge tra il giocatore che attacca e quello che difende attraverso il lancio dei dadi. Il numero dei dadi da lanciare è stabilito dal numero di armate che si decide di schierare in guerra meno una, fino ad un massimo di tre dadi per volta. Ognuno dei giocatori lancia il numero di dadi corrispondenti, e poi si confrontano i valori ottenuti, il più alto dell'attaccante con il più alto del difensore, il secondo con il secondo e così via. Per ogni punteggio più alto, il perdente deve togliere un'armata dal tabellone. In caso di parità il punto va al difensore. Il lancio dei dadi viene simulato mediante la generazione di numeri pseudocasuali.

Se lo stato attaccato perde tutte le armate, questo viene conquistato e vengono spostate automaticamente in esso un numero di armate pari a quelle sopravvissute all'attacco.

Un apposito timer determina quando il tempo per svolgere un'azione da parte di un giocatore scade e il turno passa quindi al giocatore successivo.
Al termine del turno, se è stata svolta almeno un'azione, il giocatore riceve un numero di carri armati da posizionare pari al numero di stati posseduti diviso tre, arrotondato per eccesso.
Un'apposita procedura consente al client di sapere, quando è il turno del giocatore, tutto lo stato di gioco e quindi far scegliere quale azione effettuare. Si ricorda, comunque, che tutta la logica applicativa è implementata nel DBMS.
Un giocatore può sempre visualizzare lo storico di tutte le partite giocate.

2. Analisi dei Requisiti

Lo scopo di questa sezione è raffinare la specifica fornita, andando ad effettuare un'operazione preliminare di disambiguazione.

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
8	amministratori	moderatori	Termine ambiguo che può dar luogo a incomprensioni, si preferisce usare sempre il vocabolo 'moderatore'
21, 32, 36	Numero di armate	Numero di carri armati	Vengono usati termini differenti per lo stesso significante
23, 27	Giocatore che attacca	Attaccante	Termini differenti per individuare lo stesso giocatore
24, 28	Giocatore che difende	Difensore	Termini differenti per individuare lo stesso giocatore
33	Un apposito timer determina [...]	Un apposito timer dal valore di due minuti [...]	Espressione poco specifica, di preferisce introdurre fin da subito una grandezza quantificabile

Specifica disambiguata

Sistema di gioco Risiko online

Si vuole realizzare un servizio online che consenta di giocare ad un clone del famoso gioco Risiko, in modalità "conquista del mondo".

Al sistema hanno accesso due tipologie di utenti: i giocatori e i moderatori. I moderatori hanno la possibilità di creare stanze di gioco, in funzione della quantità di giocatori che utilizzano attualmente il sistema. In particolare, i moderatori hanno la possibilità di visualizzare, tramite un report, quante stanze hanno attualmente partite in corso e quanti giocatori stanno partecipando alla partita. Inoltre, in questo report, i moderatori possono visualizzare il numero totale di giocatori che hanno effettuato almeno un'operazione negli ultimi 15 minuti che non sono all'interno di alcuna stanza di gioco.

Una stanza permette ad un numero massimo di sei giocatori di entrare e partecipare alla partita. Una partita coinvolge almeno tre giocatori. Quando il terzo giocatore entra in una stanza, viene attivato un countdown tale da avviare la partita dopo due minuti. In questi due minuti altri giocatori possono entrare, fino al massimo concesso.

All'avvio della partita, gli stati del tabellone vengono assegnati casualmente ai giocatori. I turni "girano" in funzione del tempo di ingresso dei giocatori nella stanza (chi è entrato prima gioca prima). Un turno prevede che il giocatore possa compiere una delle seguenti azioni:

* Posizionare un numero arbitrario di carri armati in uno stato in suo possesso
* Scegliere uno stato da cui fare partire un attacco verso uno stato adiacente detenuto da un altro giocatore
* Spostare carri armati da uno stato ad un altro adiacente (almeno un carro armato deve restare nello stato di partenza) entrambi sotto il controllo del giocatore

L'attacco viene svolto nel seguente modo. La fase di attacco si svolge tra il giocatore che attacca e il giocatore che difende attraverso il lancio dei dadi. Il numero dei dadi da lanciare è stabilito dal numero di carri armati che si decide di schierare in guerra meno una, fino ad un massimo di tre dadi per volta. Ognuno dei giocatori lancia il numero di dadi corrispondenti, e poi si confrontano i valori ottenuti, il più alto dell'attaccante con il più alto del difensore, il secondo con il secondo e così via. Per ogni punteggio più alto, il perdente deve togliere un carro armato dallo stato attaccato. In caso di parità il punto va al difensore. Il lancio dei dadi viene simulato mediante la generazione di numeri pseudocasuali.

Se lo stato attaccato perde tutte le armate, questo viene conquistato e vengono spostate automaticamente in esso un numero di armate pari a quelle sopravvissute all'attacco.
Un apposito timer dal valore di due minuti determina quando il tempo per svolgere un'azione da parte di un giocatore scade e il turno passa quindi al giocatore successivo.
Al termine del turno, se è stata svolta almeno un'azione, il giocatore riceve un numero di carri armati da posizionare pari al numero di stati posseduti diviso tre, arrotondato per eccesso.
Un'apposita procedura consente al client di sapere, quando è il turno del giocatore, tutto lo stato di gioco e quindi far scegliere quale azione effettuare. Si ricorda, comunque, che tutta la logica applicativa è implementata nel DBMS.
Un giocatore può sempre visualizzare lo storico di tutte le partite giocate.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Utente	Generico fruitore del gioco		Giocatore, moderatore
Giocatore	Utente generico che gioca al Risiko online caratterizzato da un ID univoco che può sia partecipare ad una partita entrando in una stanza sia svolgere altre operazioni	Attaccante, difensore	Stanza, partita, turno, utente
moderatore	Utente in grado di accedere a più dati sul DB e di creare le stanze		Stanza, report, utente

Stanza	Stanza di gioco creata da un moderatore ad un determinato timestamp in cui i giocatori se in numero compreso tra tre e sei possono partecipare ad una partita	Stanza di gioco	Moderatore, giocatore, partita
Carro armato	Attributo che permette lo sviluppo della fase di attacco e difesa del gioco	armata	
Partita	Indica una partita generica che si svolge all'interno di una stanza se ci sono abbastanza giocatori		stanza, turno,giocatore
turno	Turno in cui i giocatori possono compiere azioni relative alle fasi di una partita	Round, turno di gioco	Partita, giocatore, azione
azione	È un'operazione che compie il giocatore durante il suo turno in una partita	Azione di gioco	Turno, giocatore, partita
stato	Uno dei 42 stati rappresentati nel Tabellone di gioco	Territorio, nazione	Giocatore, partita, azione
Dadi	Indica l'operazione di lancio dei dati utilizzata per definire il risultato di un attacco	Lancio dei dadi	Azione, giocatore, turno, partita
report	Report con i risultati di tutte le partite giocate se richiesto da un giocatore, se richiesto da un'utenza moderatore visualizza il numero di stanze con partite in corso e il numero totale di giocatori in partita e non.	storico	Moderatore, giocatore

Raggruppamento dei requisiti in insiemi omogenei

Frasi relative a 'giocatore'
Al sistema hanno accesso due tipologie di utenti: I giocatori e I moderatori Una partita coinvolge almeno tre giocatori, Quando il terzo giocatore entra in una stanza, viene attivato un countdown tale da avviare la partita dopo due minuti. In questi due minuti altri giocatori possono entrare, fino al massimo concesso. il giocatore possa compiere una delle seguenti azioni: * Posizionare un numero arbitrario di carri armati in uno stato * Scegliere uno stato da cui fare partire un attacco verso uno stato adiacente * Spostare carri armati da uno stato ad un altro adiacente (almeno un carro armato deve restare nello stato di partenza) La fase di attacco si svolge tra il giocatore che attacca e quello che difende attraverso il lancio dei dadi. Ognuno dei giocatori lancia il numero di dadi corrispondenti, e poi si confrontano i valori ottenuti, […] Determina quando il tempo per svolgere un'azione da parte di un giocatore scade e il turno passa quindi al giocatore successivo. Il giocatore riceve un numero di carri armati da posizionare pari al numero degli stati posseduti diviso tre arrotondato per eccesso. Un giocatore può sempre visualizzare lo storico di tutte le partite giocate.
Frasi relative a 'moderatore'
Al sistema hanno accesso due tipologie di utenti: I giocatori e I moderatori I moderatori hanno la possibilità di creare stanze di gioco, in funzione della quantità di giocatori che utilizzano attualmente il sistema. i moderatori hanno la possibilità di visualizzare, tramite un report, quante stanze hanno attualmente partite in corso e quanti giocatori stanno partecipando alla partita. Inoltre, in questo report, gli amministratori possono visualizzare il numero totale di giocatori che hanno effettuato almeno un'operazione negli ultimi 15 minuti che non sono all'interno di alcuna stanza di gioco.
Frasi relative a 'stanza'
I moderatori hanno la possibilità di creare stanze di gioco gli amministratori possono visualizzare il numero totale di giocatori che hanno effettuato almeno un'operazione negli ultimi 15 minuti che non sono all'interno di alcuna stanza di gioco. Una stanza permette ad un numero massimo di sei giocatori di entrare e partecipare alla partita. Quando il terzo giocatore entra in una stanza, viene attivato un countdown
Frasi relative a 'partita'

Una stanza permette ad un numero massimo di sei giocatori di entrare e partecipare alla partita.

Una partita coinvolge almeno tre giocatori.

Quando il terzo giocatore entra in una stanza, viene attivato un countdown tale da avviare la partita dopo due minuti.

All'avvio della partita, gli stati del tabellone vengono assegnati casualmente ai giocatori.

Un giocatore può sempre visualizzare lo storico di tutte le partite giocate.

Frasi relative a 'turno'

I turni "girano" in funzione del tempo di ingresso dei giocatori nella stanza (chi è entrato prima gioca prima).

Un turno prevede che il giocatore possa compiere una delle seguenti azioni: […]

Un apposito timer determina quando il tempo per svolgere un'azione da parte di un giocatore scade e il turno passa quindi al giocatore successivo.

Al termine del turno, se è stata svolta almeno un'azione, il giocatore riceve un numero di carri armati da posizionare pari al numero di stati posseduti diviso tre, arrotondato per eccesso.

Un'apposita procedura consente al client di sapere, quando è il turno del giocatore, […]

Frasi relative ad 'azione'

* Posizionare un numero arbitrario di carri armati in uno stato in suo possesso
* Scegliere uno stato da cui fare partire un attacco verso uno stato adiacente detenuto da un altro giocatore
* Spostare carri armati da uno stato ad un altro adiacente (almeno un carro armato deve restare nello stato di partenza) entrambi sotto il controllo del giocatore

Frasi relative a 'stato'

All'avvio della partita, gli stati del tabellone vengono assegnati casualmente ai giocatori.

Per ogni punteggio più alto, il perdente deve togliere un carro armato dallo stato attaccato.

Se lo stato attaccato perde tutte le armate, questo viene conquistato e vengono spostate automaticamente in esso un numero di armate pari a quelle sopravvissute all'attacco.

il giocatore riceve un numero di carri armati da posizionare pari al numero di stati posseduti diviso tre [...]

Frasi relative a 'Dadi'

La fase di attacco si svolge tra il giocatore che attacca e quello che difende attraverso il lancio dei dadi.

Il numero dei dadi da lanciare è stabilito dal numero di armate che si decide di schierare in guerra meno una, [...]

Ognuno dei giocatori lancia il numero di dadi corrispondenti, e poi si confrontano i valori ottenuti, il più alto dell'attaccante con il più alto del difensore, il secondo con il secondo e così via. Il lancio dei dadi viene simulato mediante la generazione di numeri pseudocasuali.
Frasi relative a 'report'
Un giocatore può sempre visualizzare lo storico di tutte le partite giocate.

3. Progettazione concettuale

Costruzione dello schema E-R

Si è deciso di adottare una strategia inside-out, ovvero a macchia d'olio, in quanto nonostante si avessero fin da subito tutte le specifiche di progetto, a meno di disambiguazioni, si è preferito realizzare il diagramma ER partendo da concetti centrali ed ampliarlo iterativamente aggiungendo ogni volta una o più nuove entità verificando la correttezza delle relazioni esistenti tra esse al fine di non dover compiere al termine del processo di costruzione verifiche e modifiche complesse ed onerose. In alcuni passaggi sono stati omessi volontariamente degli attributi e le cardinalità sono state aggiunte solo nelle ultime fasi di sviluppo in quanto si è preferito procedere con un diagramma il più leggero e leggibile possibile.

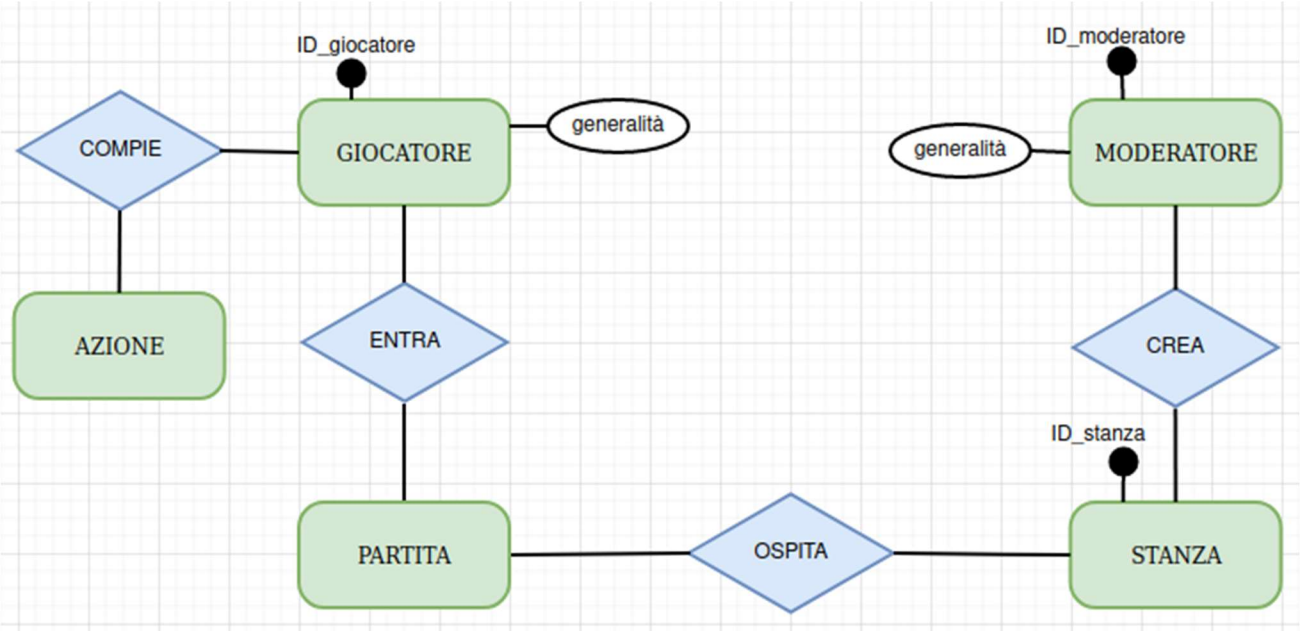


Fig. 1

Dallo schema in fig. 1 si nota come le entità GIOCATORE e MODERATORE condividano degli attributi e siano entrambi contraddistinti da un codice identificativo. Questa breve analisi ci permette di introdurre l'entità generale UTENTE.
Viene inoltre previsto in questo primo schema la rappresentazione dell'entità AZIONE che, come si vede dal raggruppamento dei requisiti in insiemi omogenei, comprende le tre azioni possibili specificate nella consegna del progetto e fa riferimento al concetto di TURNO che a sua volta è in relazione con PARTITA. Procediamo integrando i nuovi concetti appena introdotti e applicando la generalizzazione sull'utenza presentata sopra.

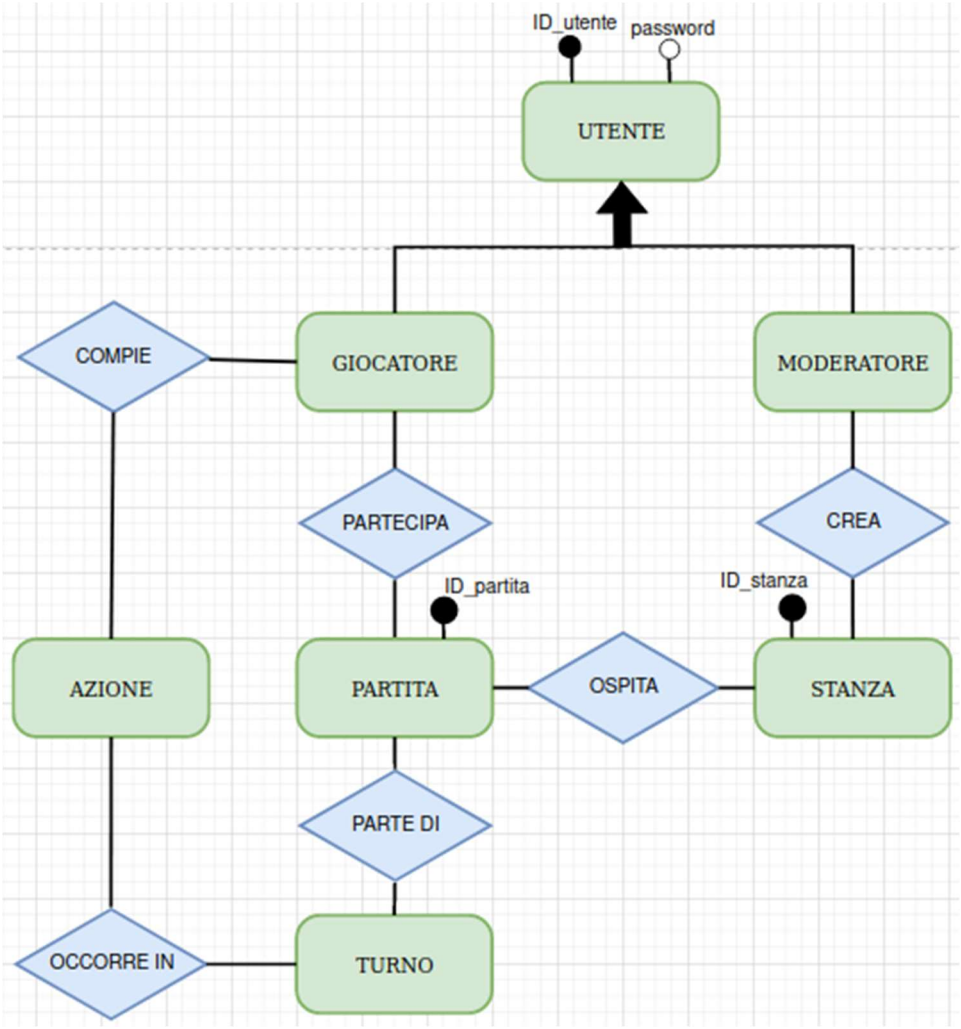


Fig. 2

Siamo pronti per introdurre nuovi concetti e dato che lo schema comincia a diventare più complesso, in questa iterazione introduciamo tutti gli attributi e gli identificatori delle entità.

Nel risultato illustrato in fig. 3 sono stati inseriti:

- Attributi e identificatori relativi a tutti i concetti fin qui introdotti

- L’entità TERRITORIO, le relative relazioni con gli altri concetti nello schema ER e gli attributi ‘nome’ e ‘carri armati presenti’ per specificare il nome del Territorio e i carri armati attualmente posizionati.

Per quanto riguarda il legame con le altre entità TERRITORIO si relaziona sia con l’entità GIOCATORE in quanto come da regolamento, ad ogni inizio partita vengono associati dei territori ad ogni giocatore ed un giocatore può occuparne altri durante una partita; sia con l’entità PARTITA in quanto un TERRITORIO esiste solo all’interno di una PARTITA.

Notiamo la necessità di specificare l'adiacenza tra territori fondamentale per svolgere tutte e tre le azioni di gioco. Tale proprietà vale indipendentemente dalla partita in cui le nazioni sono rappresentate e ci porta a considerare l’inserimento di una nuova entità per rappresentare l’astrazione dei Territori dalle loro istanze nelle singole partite.

Si nota infine che il legame tra turno e azione probabilmente si semplificherà visti i pochi attributi e la stretta correlazione tra l’esistenza delle due entità, per il momento lasciamo entrambe le entità. Tutto ciò sarà oggetto del prossimo step.

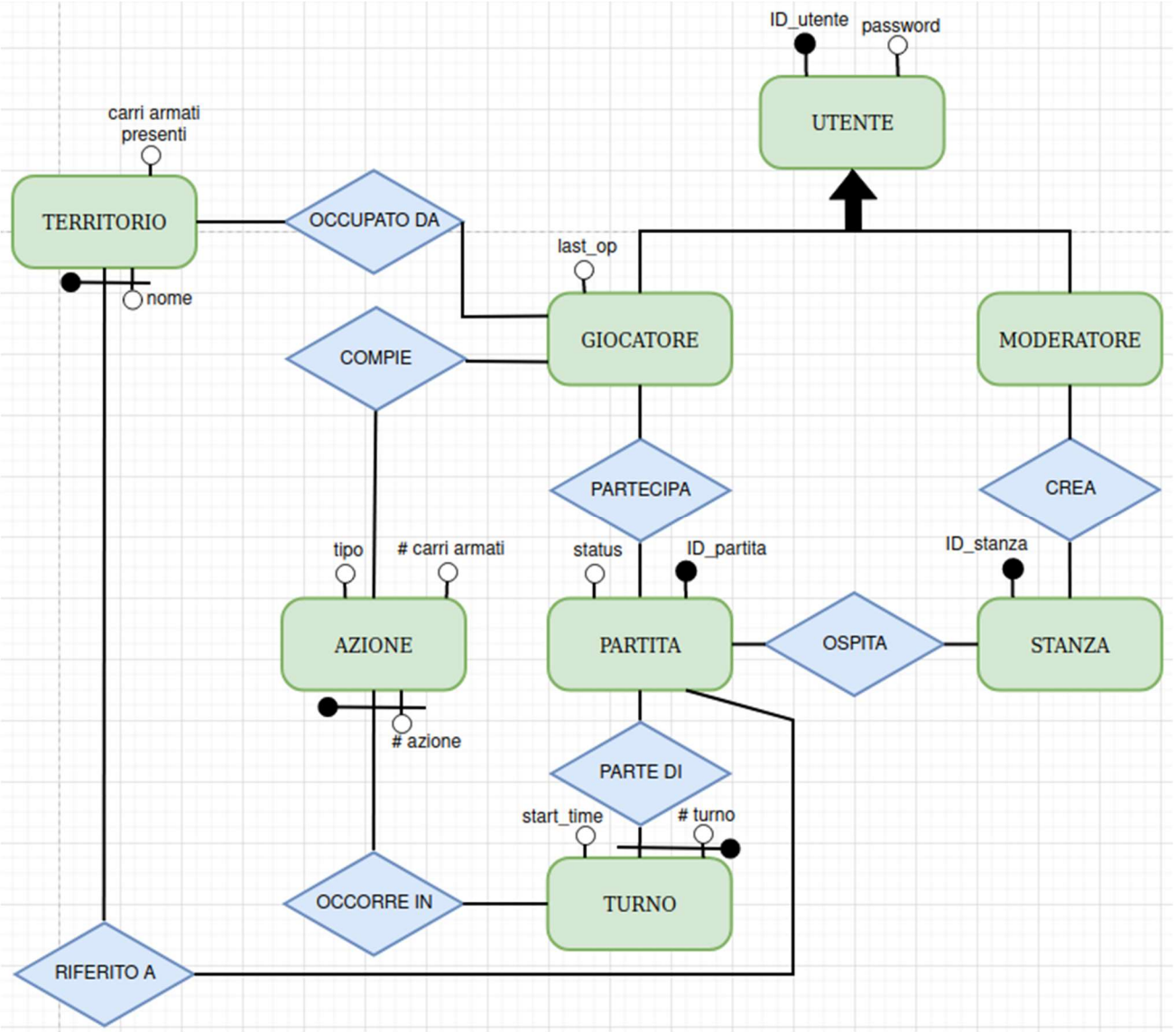


Fig. 3

Andiamo ad introdurre come mostrato in *fig.4* l'entità NAZIONE in quanto le caratteristiche intrinseche discusse sopra ci portano a dover differenziare il concetto astratto dalle singole istanze create per ogni partita, per questo si introduce l'entità NAZIONE e di conseguenza varia anche l'identificatore di TERRITORIO visto che l'attributo 'nome' è proprio di una delle 42 Nazioni del *Risiko.* Inseriamo anche la relazione ricorsiva ADIACENZA dato che ogni singola NAZIONE confina con almeno un'altra.

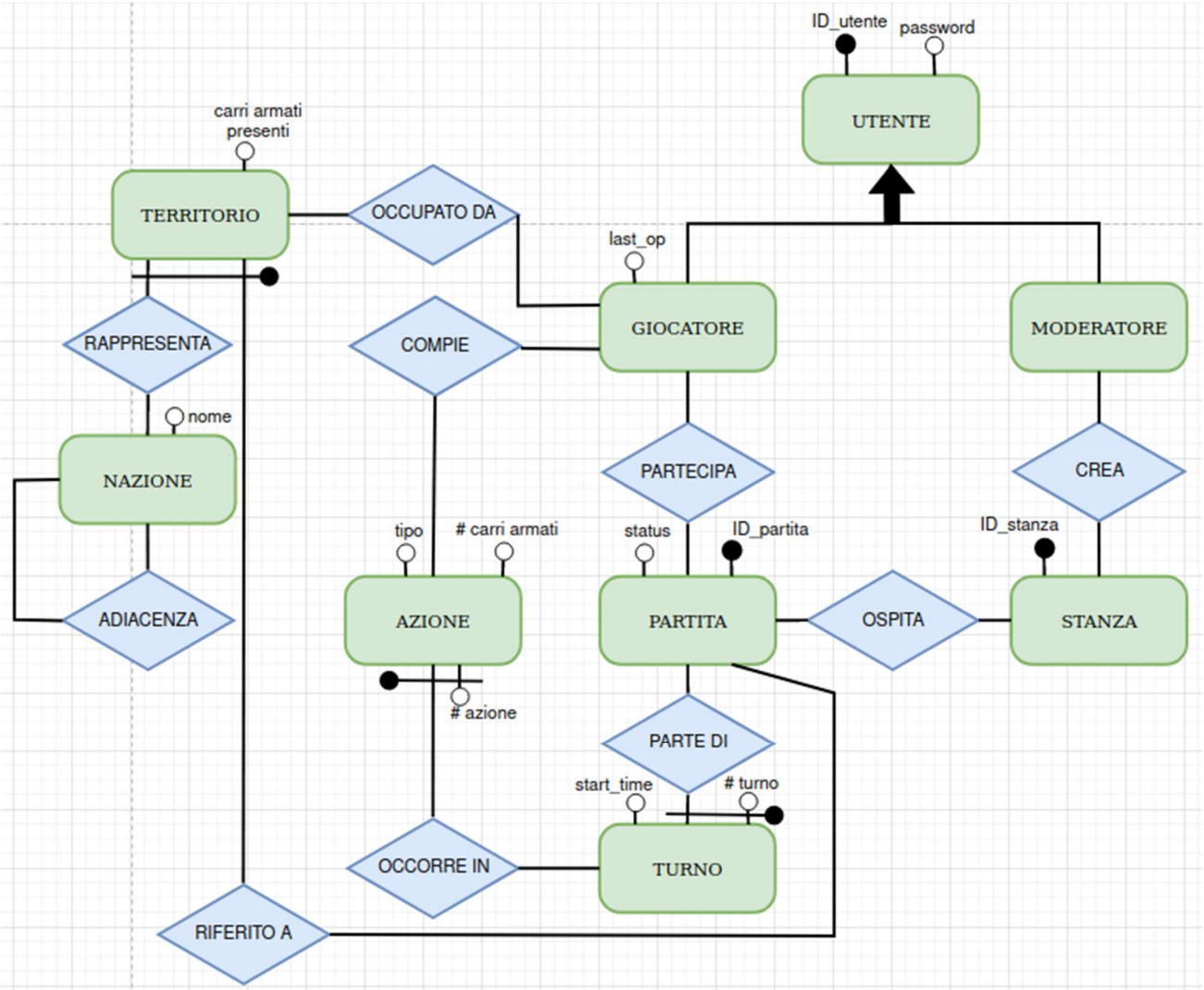


Fig. 4

Il diagramma comincia ad essere rappresentativo delle specifiche fornite, nel prossimo passo, illustrato in fig.5, si andranno ad introdurre le cardinalità e si approfondirà il legame tra GIOCATORE e TURNO.

Per quanto riguarda le cardinalità partiamo dalla prima entità in alto a sinistra e scorriamo il diagramma per righe.

- TERRITORIO: può essere occupato da un solo GIOCATORE e di conseguenza avrà cardinalità (1,1) per la relazione OCCUPATO DA e risulta essere un'entità debole verso NAZIONE, pertanto anche sulla relazione con quest'ultima avrà cardinalità (1,1). Infine, ogni territorio è associato univocamente ad una partita ed ogni PARTITA ha associati al massimo

42 TERRITORI e al minimo 0 qualora la partita non sia ancora incominciata, le cardinalità sulla relazione sono quindi, rispettivamente (1,1) e (0, 42).

- GIOCATORE: Tale entità si relaziona con TERRITORIO in quanto un giocatore ne può occupare molteplici, da zero nel caso in cui abbia perso la partita fino al caso remoto in cui conquisti tutti i territori del gioco. Altra entità con cui si relaziona il giocatore è PARTITA e come nel caso precedente la cardinalità risulta essere (0, N).
- MODERATORE: Un moderatore da specifiche si occupa di creare un numero di stanze adeguato ai giocatori online che potrebbero essere zero come molti, questo si traduce con cardinalità (0, N).
- AZIONE: Risulta essere un'entità debole rispetto a TURNO, quindi avrà cardinalità (1,1) e (0,N) per quanto riguarda la relazione OCCORRE IN, rispetto a GIOCATORE avrà cardinalità (1,1) e (0,N) in quanto una azione può appartenere ad un solo giocatore che nel corso della partita può compierne tante quanti sono i turni che gioca.
- PARTITA: Da regolamento una PARTITA è composta da massimo sei giocatori, ciò si traduce con cardinalità (0,6) sulla relazione PARTECIPA. Una PARTITA si svolge tramite dei TURNI che possono essere zero qualora non si giochi fino ad N nel caso di una partita complessa. Inoltre, non possono esistere più partite in contemporanea ospitate all'interno della stessa STANZA: la cardinalità della relazione sarà anche qui (1,1) visto che è obbligatorio per una PARTITA appartenere ad una STANZA.
- STANZA: è creata da uno ed un solo MODERATORE e può ospitare da zero a molte partite purché non in contemporanea, dunque, si relaziona con cardinalità (1,1) verso MODERATORE e cardinalità (0, N) verso PARTITA.
- TURNO: I turni compongono l'entità PARTITA e devono essere parte di almeno una di esse e riconducili al massimo ad una sola PARTITA, ne segue che la cardinalità della relazione è (1,1). Durante un TURNO un giocatore può compiere o non compiere azioni, in tal caso la cardinalità sulla relazione di occorrenza è opzionale. Infine, con l'introduzione della nuova relazione tra GIOCATORE e TURNO consideriamo che un TURNO è associabile ad un solo giocatore e non è ripetibile nel tempo ed un GIOCATORE invece può giocare zero o molteplici turni, pertanto le cardinalità saranno rispettivamente (1,1) e (0, N).

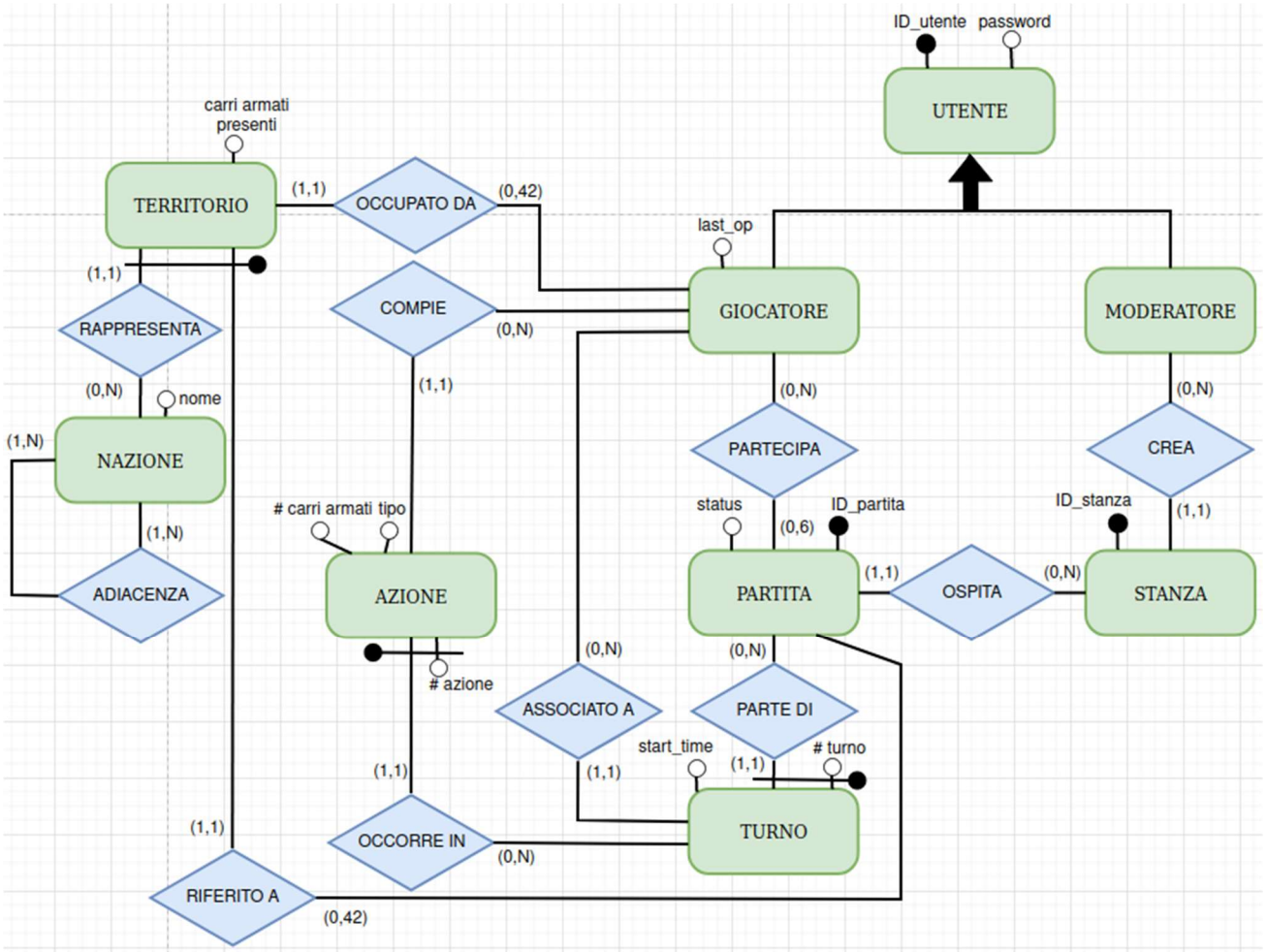


Fig. 5

Integrazione finale

In questa ultima fase andiamo a rivedere tutto il diagramma ER e nel dettaglio i vari attributi oltre a rivedere la possibile unione tra le entità TURNO e AZIONE di cui discusso in precedenza.

Nel dettaglio, visto che abbiamo già previsto l'attributo 'status' per descrivere i vari stati possibili di una PARTITA la scelta più naturale appare essere l'introduzione di alcuni attributi sulla relazione PARTECIPA per definire quando il GIOCATORE prende parte alla partita e di conseguenza in quale status della stessa si trova a partecipare. Considerato che può uscire sia durante il countdown, sia nella fase di attesa di altri giocatori e sia per eliminazione, aggiungiamo un attributo inizialmente impostato a 0 che specifica se il giocatore è stato eliminato (set ad 1 dell'attributo se eliminato) dalla partita. Inseriamo inoltre gli attributi 't_entrata' e 'num_turno' che ci permetteranno di definire

l'istante in cui il giocatore ha cominciato la sua *partecipazione* alla partita e definire in base a ciò, come richiesto da specifiche, il turno di gioco.
Considerata ora la relazione univoca tra TURNO ed AZIONE, visto che da specifiche un giocatore può compiere un'unica azione per turno e constatato che AZIONE è un'entità debole rispetto a TURNO, si decide di integrare i pochi attributi di AZIONE in TURNO e GIOCATORE eliminando *'# azione'* ormai superfluo. Nello specifico GIOCATORE si arricchisce dell'attributo '*armate disponibili'* fondamentale per conoscere la quantità di carri armati che vengono assegnate ad ogni giocatore ad inizio partita e TURNO dell'attributo *'tipo_azione'* per rappresentare una o nessuna delle tre azioni che il giocatore può compiere durante il proprio turno.

In conclusione, non essendo presenti conflitti sui nomi, risolvendo quanto appena descritto si giunge al seguente schema ER (Fig.6).

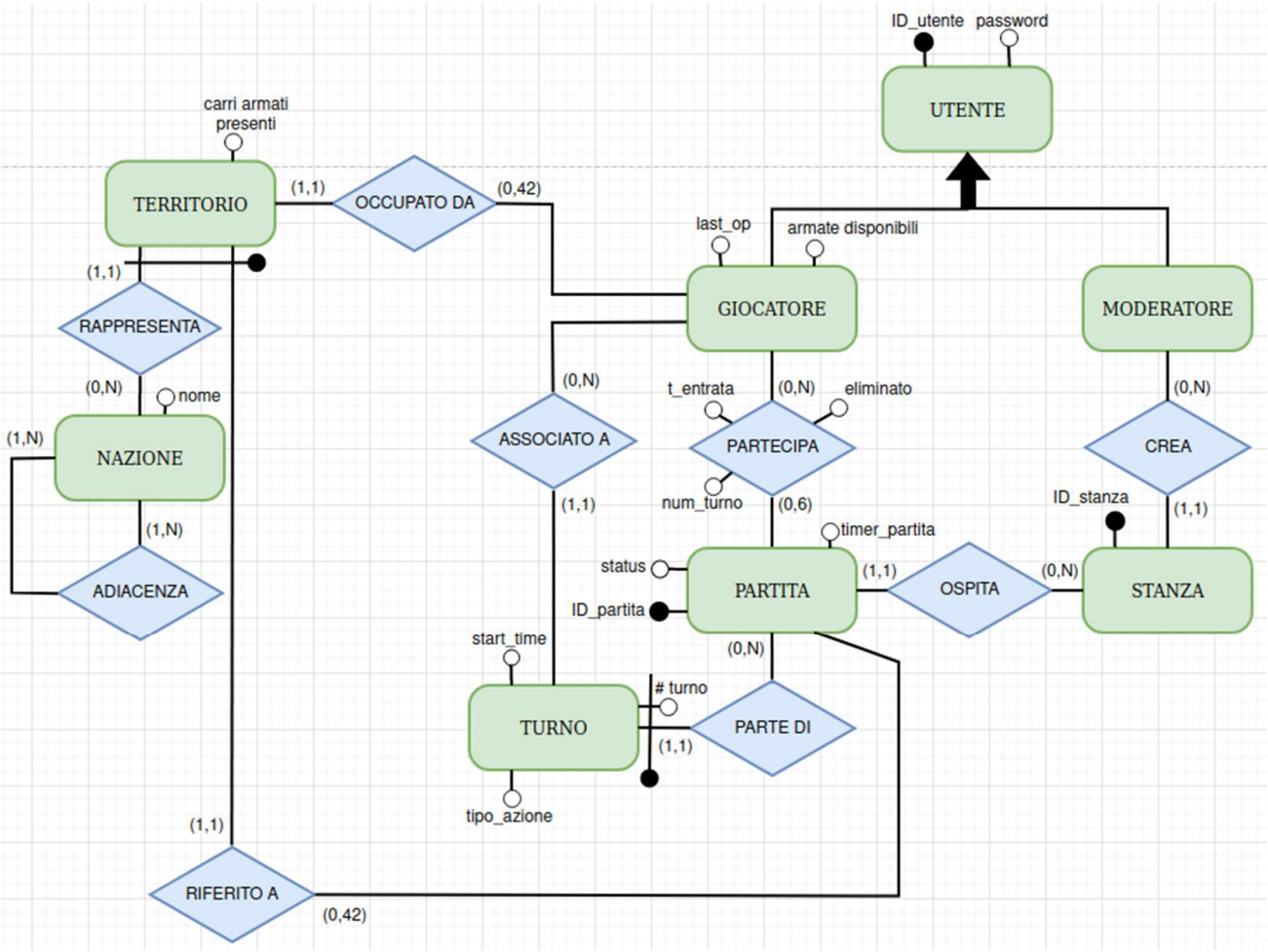


Fig. 6

Regole aziendali

Per migliorare la leggibilità si è deciso di fare una "*group by"* delle regole aziendali per *vincolo* e per *descrizione e derivazione.*

Regole aziendali di vincolo:

RA1) Un giocatore può partecipare al massimo ad una sola partita con attributo *'status'* pari a 'in_corso'

RA2) Il timer del turno deve avere una durata di 2 minuti

RA3) Ogni partita in cui partecipano almeno tre giocatori deve avviarsi dopo 2 minuti dal tempo del timer della partita

RA4) Ogni partita deve avere associati 42 Territori che riflettono lo stato del tabellone di gioco

RA5) Su ogni Territorio sotto il controllo del giocatore deve essere presente almeno un carro armato

RA6) L'attributo *'tipo_azione'* deve assumere uno tra i seguenti valori: 'rinforzo', 'spostamento', 'attacco', 'no_azione'

RA7) L'attributo *'status'* deve assumere uno tra i seguenti valori: 'in_attesa', 'in_corso', 'terminata'

RA8) L'attributo *'num_turno'* deve essere al massimo pari a 6

Regole aziendali di descrizione e derivazione:

RA9) Il time-out di un turno si rileva quando si ottiene una durata del turno maggiore del tempo di 2 minuti previsto dal timer

RA10) Il termine di una partita si ottiene quando tutti i giocatori tranne uno sono stati eliminati

RA11) Il vincitore di una partita è l'ultimo giocatore presente in partita che possiede tutti e 42 i Territori

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Stanza	Rappresenta il luogo virtuale in cui i giocatori possono prendere parte ad una partita	ID_stanza,	ID_stanza
partita	Rappresenta una generica partita di *Risiko*	ID_partita, status_partita, timer_partita	ID_partita
giocatore	Utente che gioca al Risiko online caratterizzato da un ID univoco che può sia partecipare ad una partita entrando in una stanza sia svolgere altre operazioni	last_op, armate_disponibili	ID_utente
moderatore	Utente in grado di creare stanze ed estrarre un report dal sistema con le	ID_utente	ID_utente

	informazioni relative alle operazioni svolte dai giocatori negli ultimi 15 minuti		
Utente	Generico utente del sistema, generalizzazione di giocatore e moderatore	ID_utente, password	ID_utente
Turno	Generico turno durante una partita	#_turno, start_time, tipo_azione	#_turno, ID_partita
Nazione	Una delle 42 nazioni del gioco	nome	nome
Territorio	Rappresenta una Nazione durante una singola partita	carri armati presenti	nome, ID_partita

4. Progettazione logica

Volume dei dati

Basandosi sullo studio di piattaforme che erogano lo stesso servizio già presenti sul mercato[1] si considera un'utenza massima di 10.000 giocatori. Contando che ogni moderatore crea al più 4 stanze per gestire tutti i giocatori sono necessari non più di 500 moderatori e gli utenti totali si attestano sui 10.500.

Basandosi su dati simil realistici si avranno tante partite almeno quanti sono i giocatori considerato che tutti ne vogliono vincere almeno una. A ogni partita prendono parte in media 4 giocatori e per conquistare tutti e 42 i Territori, esclusi quelli già assegnati in fase di avvio della partita ad ogni giocatore, servono ad un giocatore mediamente bravo circa 50 turni, ne segue che la partita media sarà composta da 200 turni.

Concetto nello schema	Tipo[2]	Volume atteso
Giocatore	E	10.000
Moderatore	E	500
Utente	E	10.500
Stanza	E	2.000 (4 stanze per moderatore)
partita	E	100.000
Turno	E	20.000.000 (200 turni a partita)
Nazione	E	42
Territorio	E	420.000
Adiacenza	R	123

[1] Risiko! Digital 3 – EDITRICE GIOCHI

[2] Indicare con E le entità, con R le relazioni

Rappresenta	R	420.000
Occupato da	R	420.000
partecipa	R	100.000
crea	R	2.000
ospita	R	100.000
associato	R	20.000.000
Parte di	R	100.000
Riferito a	R	420.000

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
LU1	Eseguire il login	525 per day
MR1	Report del moderatore con partite in corso e giocatori in stanza	25 per day
MC1	Creare una stanza	100 per month
GR1	Visualizzare il report con lo storico delle partite a cui ha preso parte un giocatore	800 per day (un 60% dei giocatori giornalieri guarda almeno due volte il report)
GS1	Partecipare ad una partita	1000 per day (ogni giocatore in un giorno partecipa ad almeno 2 partite)
GA1	Posizionare dei carri armati su un territorio	40.000 per day (200 turni * 1000 partite) * 20%
GA2	Spostare dei carri armati tra territori adiacenti	60.000 per day (200 turni * 1000 partite) * 30%
GA3	Attaccare un territorio	80.000 per day (200 turni * 1000 partite) * 40%
GA4	Visualizzare lo stato della partita	160.000 per day (200 turni* 1000 partite) * 80%
ST1	Saltare un turno	20.000 per day (200 turni * 1000 partite) * 10%

Costo delle operazioni

Si suppone che il costo in scrittura di un dato sia doppio rispetto a quello in lettura.

LU1 - Eseguire il login

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Utente	E	1	L

Costo totale: 1

Accessi/day = 1*525 = 525

MR1 - Report partite in corso e giocatori in stanza

Ci si pone nel caso in cui l'80% delle stanze totali abbiano una partita in corso, per un totale di 1.600 partite in contemporanea e, considerata una media di 4 giocatori a partita, 6.400 giocatori partecipanti. Tale stima si avvicina all'upper bound rappresentato dall'avere tante partite in contemporanea quante sono le stanze. L'accesso viene eseguito su tutte le istanze di 'Giocatore' in quanto è necessario da specifica considerare sia i giocatori in partita sia i giocatori attivi che abbiano compiuto una operazione negli ultimi 15 minuti.

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Stanza	E	2.000	L
Partita	E	1.600	L
Giocatore	E	10.000	L
Partecipa	R	6.400	L

Costo totale: 2000 + 1600 + 10000 + 6400 = 20.000

Accessi/day: 20.000*25 = 500.000

MC1 – Creare una stanza

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Stanza	E	1	S
Partita	E	1	S

Costo totale: 1*2 + 1*2 = 4

Accessi/month: 4*100 = 400

GR1 - Visualizzare il report con lo storico delle partite a cui ha preso parte un giocatore

Si suppone che un giocatore partecipi in media a 2 partite al giorno

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Partecipa	R	2	L
Partita	E	2	L

Costo totale: 2+2 = 4

Accessi/day: 4*800 = 3.200

GS1 – Partecipare ad una partita

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Partecipa	R	1000	S

Costo totale: 2

Accessi/day: 2*1000 = 2.000

GA1 - Posizionare dei carri armati su un territorio

In questa operazione un giocatore va a posizionare un certo numero di carri armati su un Territorio sotto il suo dominio. Requisiti minimi sono quindi la disponibilità di carri armati, che dobbiamo andare a verificare leggendo il corrispondente attributo, e che il Territorio su cui si vuole effettuare il *'displacement'* sia effettivamente occupato dal giocatore, infine va considerata la scrittura dell'attributo di Turno per specificare il 'tipo' di azione che è appena stata compiuta e le due scritture per aggiornare gli attributi '*armate disponibili'* e '*carri armati presenti'* .

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Giocatore	E	2	L+S
Occupato da	R	1	L
Territorio	E	2	L+S
Associato a	R	1	L
Turno	E	1	S

Costo totale: = 1+2+1+1+2+1+2 = 10

Accessi/day: 10 * 20.000 = 200.000

GA2 - Spostare dei carri armati tra territori adiacenti

Anche per tale operazione è necessario svolgere delle verifiche a priori andando a leggere le relazioni OCCUPATO DA e ADIACENZA per assicurarci che la richiesta sia legale e scrivere il tipo di azione che è stata effettuata nel Turno associato al giocatore.

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Occupato da	R	2	L
Territorio	E	2	L-S
Adiacenza	R	82	L
Associato a	R	1	L
Turno	E	1	S

Costo totale: 2+1*2+2*2+82+1+1*2 = 93

Accessi/day: 93*30.000 = 2.790.000

GA3 – Attaccare un territorio

*La scrittura per la relazione 'occupato da' va eseguita solo nel caso in cui il giocatore che attacca conquisti il territorio del giocatore che difende, ovviamente possiamo anche riscrivere dopo l'esito di tutte le azioni di attacco, indipendentemente dal loro esito, svolgendo così operazioni evitabili. Si considera quindi un 50% di attacchi andati a buon fine. Inoltre, va considerata anche qui la scrittura dell'attributo di Turno per specificare il 'tipo' di azione.

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Territorio	E	2	L
Occupato da	R	2	L
Occupato da	R	*	S
Associato a	E	1	L
Turno	R	1	S

Costo totale: 2+2+0,5+1+2 = 5,5

Accessi/day: 5,5*40.000 = 220.000

GA4 - Visualizzare lo stato della partita
È una operazione di sola lettura che tuttavia risulta essere una delle più onerose data la grande quantità di informazioni da reperire e l'alta frequenza con cui viene eseguita.

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Giocatore	E	1	L
Occupato da	R	42	L
Territorio	E	42	L

Costo totale: 1 + 42 + 42 = 85

Accessi/day: 85*80.000 = 6.800.000

ST1– Saltare un turno
Come nelle operazioni GA1/2/3 anche qui va considerata la scrittura dell'attributo di Turno per specificare il 'tipo' di azione che risulta essere *'no_azione'*.

CONCETTO	COSTRUTTO	ACCESSI	TIPO
Turno	E	1	L
Parte di	R	1	L
Partita	E	1	L
Turno	E	1	S

Costo totale: 1+1+1+2 = 5

Accessi/day: 5*20.000 = 100.000

Ristrutturazione dello schema E-R

Per la generalizzazione UTENTE vediamo che le due entità figlie non differiscono di molto ed in particolare MODERATORE non presenta attributi specifici, l'ipotesi più plausibile risulta essere l'accorpamento delle due entità figlie nel genitore prevedendo il nuovo attributo 'tipo' che permette di discernere l'utenza in moderatori e giocatori ed eliminando la relazione 'crea' in quanto ci interessa l'atto in sé della creazione delle stanze non chi lo compie. In figura 7 il diagramma ER finale.

Con l'occasione si rinomina l'attributo di UTENTE '*last_op*' in *op_time*' per rendere più chiaro il fatto che si tratti di un istante temporale.

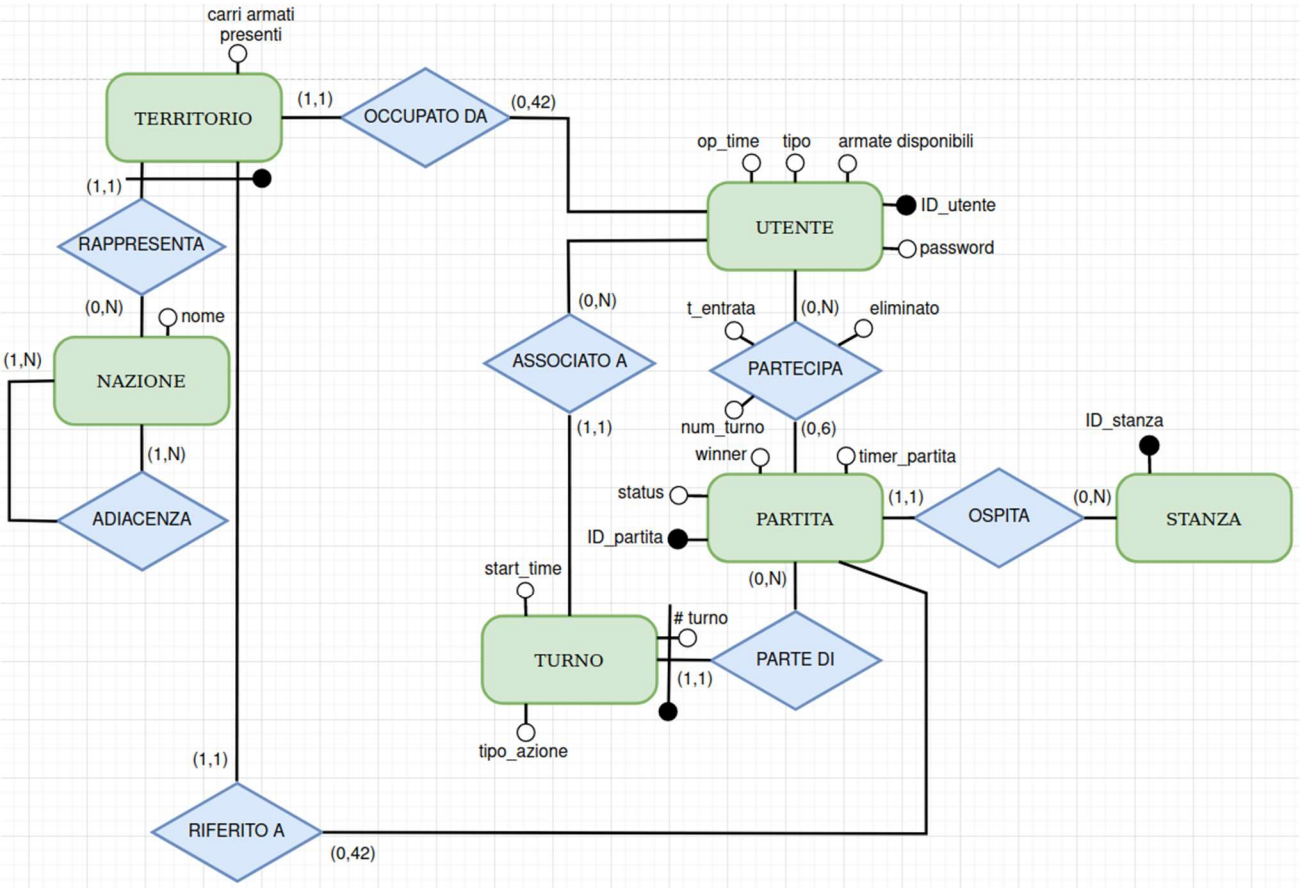


Fig. 7

Traduzione di entità e associazioni

Si riportano di seguito le varie relazione con i relativi vincoli referenziali.

NAZIONE (nome)

ADIACENZA (nazione_1, nazione_2)

- ADIACENZA (nazione_1) ⊆ NAZIONE (nome)
- ADIACENZA (nazione_2) ⊆ NAZIONE (nome)

TERRITORIO (nazione, partita, carri_armati_presenti, occupante)

- TERRITORIO (partita) ⊆ PARTITA (IDpartita)
- TERRITORIO (occupante) ⊆ UTENTE (IDutente)

UTENTE (IDutente, tipo, password, armate_disponibili, op_time)

PARTECIPA (giocatore, partita, t_entrata, eliminato, num_turno)

- PARTECIPA (giocatore) ⊆ UTENTE (IDutente)
- PARTECIPA (partita) ⊆ PARTITA (IDpartita)

PARTITA (IDpartita, stanza, status_partita, timer_partita, winner)

- PARTITA (stanza) ⊆ STANZA (IDstanza)

STANZA (IDstanza)

TURNO (#_turno, partita, start_time, giocatore, tipo_azione)

- TURNO (partita) ⊆ PARTITA(IDpartita)
- TURNO (giocatore) ⊆ UTENTE(IDutente)

In figura 8 una rappresentazione grafica delle tabelle ottenuta tramite MySQL Workbench.

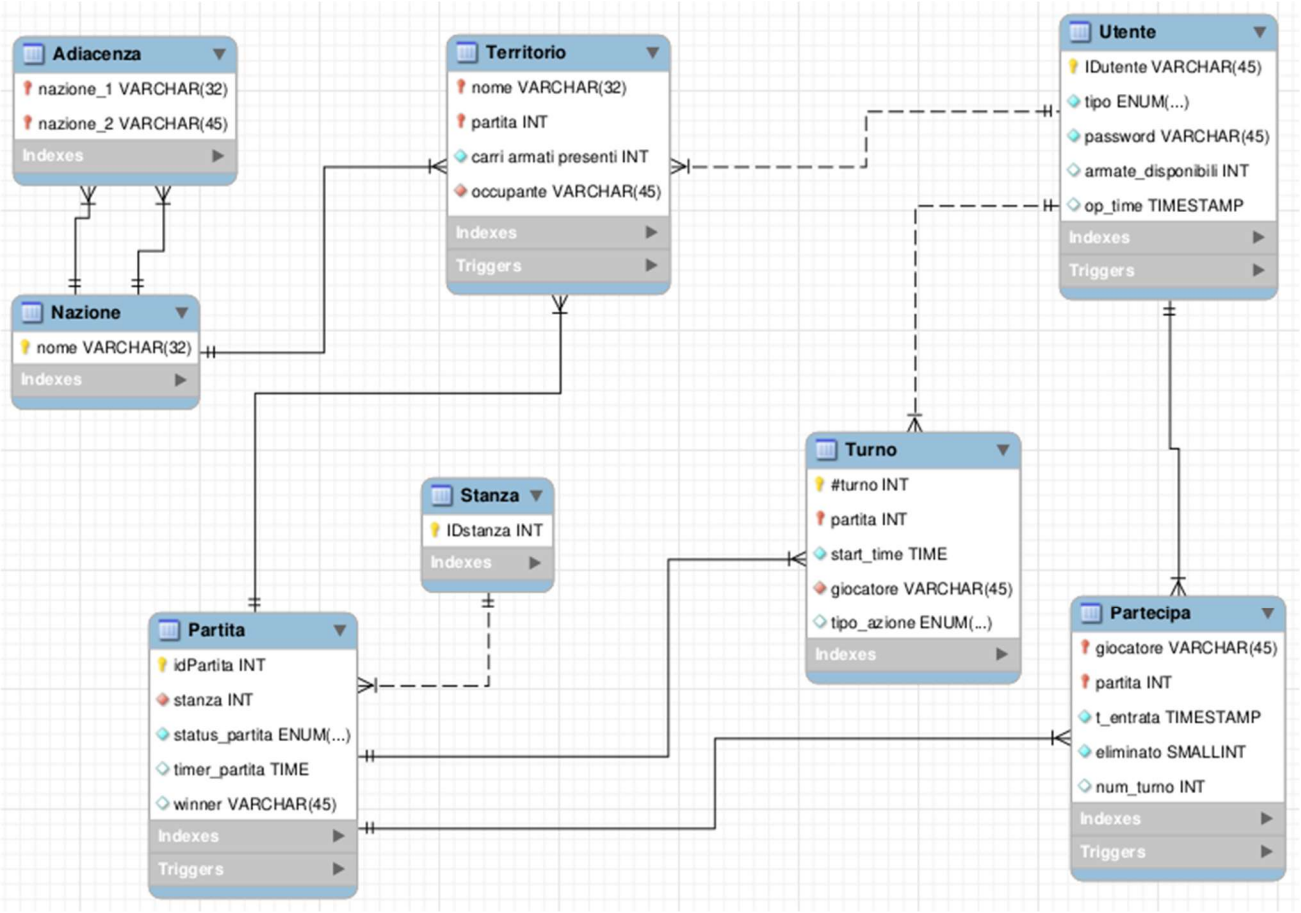


Fig. 8

Normalizzazione del modello relazionale

Per tutte le tabelle non sono definite dipendenze transitive in quanto considerata la scrittura X→A, X è sempre parte della chiave primaria e/o A appartiene ad almeno una chiave della relazione. Quindi tutte le relazioni sono in 3NF.

5. Progettazione fisica

Utenti e privilegi

Al fine di proteggere i dati mantenuti nel DBMS sono state previste tre tipologie di utenza con cui è possibile eseguire diverse procedure in base al proprio livello di privilegio:

1. "**login**" utenza che ha accesso solamente all'esecuzione della stored procedure utilizzata per eseguire il login al sistema e alla procedura per registrarsi al sistema come giocatore nel caso di nuovo accesso.

2. "**moderatore**" utenza riservata ai moderatori del gioco che garantisce l'esecuzione della stored procedure per creare una nuova stanza di gioco, quella per richiedere il report moderatore e per registrare un nuovo moderatore nel database.
3. "**giocatore**" utenza per i giocatori che permette di eseguire esclusivamente le stored procedures funzionali alle operazioni indicate con G** nella tavola delle operazioni.

Strutture di memorizzazione

Tabella Adiacenza		
Colonna	**Tipo di dato**	**Attributi**[3]
nazione_1	VARCHAR (32)	PK, NN
nazione_2	VARCHAR (32)	PK, NN

Tabella Nazione		
Colonna	**Tipo di dato**	**Attributi**
Nome	VARCHAR (32)	PK, NN, UQ

Tabella Territorio		
Colonna	**Tipo di dato**	**Attributi**
Nome	VARCHAR(32)	PK, UQ
Partita	INT	PK,NN
Carri armati presenti	INT	NN, DEFAULT = 1
Occupante	VARCHAR (45)	NN

Tabella Turno		
Colonna	**Tipo di dato**	**Attributi**
#turno	INT	PK, NN
Partita	INT	PK, NN
start time	TIME	NN
giocatore	VARCHAR (45)	NN
Tipo_Azione	ENUM	DEFAULT = NULL

[3] PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella Partita		
Colonna	**Tipo di dato**	**Attributi**
IDpartita	INT	PK, NN
stanza	INT	NN
status_partita	ENUM	NN
timer_partita	TIME	
winner	VARCHAR (45)	

Tabella Stanza		
Colonna	**Tipo di dato**	**Attributi**
IDstanza	INT	PK, NN

Tabella Partecipa		
Colonna	**Tipo di dato**	**Attributi**
giocatore	VARCHAR (45)	PK, NN
partita	INT	PK, NN
t_entrata	TIMESTAMP	NN
eliminato	SMALLINT	NN, DEFAULT = 0
num_turno	INT	

Tabella Utente		
Colonna	**Tipo di dato**	**Attributi**
IDutente	VARCHAR (45)	PK, NN
tipo	ENUM	NN
password	VARCHAR (45)	NN
armate_disponibili	BIT	
op_time	TIMESTAMP	

Indici

Non sono previsti indici aggiuntivi rispetto a quelli generati automaticamente dal DBMS.

Trigger

Di seguito i quattro trigger che sono stati implementati.

1. Trigger before update sulla tabella ‘Utente’ per verificare se un giocatore ha a disposizione i carri armati richiesti dall’azione di ‘posizionamento’ su un Territorio.

```
CREATE DEFINER = CURRENT_USER TRIGGER
`Risiko_online`.`BU_check_armate_disponibili` BEFORE UPDATE ON `Utente` FOR EACH
ROW
```

```
BEGIN
        IF NEW.`armate_disponibili` < 1  THEN
SIGNAL SQLSTATE '45013' SET MESSAGE_TEXT = 'Numero di carri armati disponibili insufficiente'
        END IF;
END
```

2. Trigger che chiama le stored procedures per eseguire il set-up di una partita il cui status viene aggiornato da '*in_attesa'* ad '*in_corso'* e per avviare il primo della stessa.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Risiko_online`.`Partita_set_avvio` AFTER UPDATE ON `Partita` FOR EACH ROW
BEGIN
declare var_IDplayer VARCHAR(45);

   IF NEW.`status_partita` = 'in_corso' THEN
-- call a procedure per setting della partita
CALL set_partita(OLD.`IDpartita`);

      -- avviamento del primo turno
      SELECT `giocatore` FROM `Partecipa`
      WHERE `partita` = OLD.`IDpartita` AND `num_turno` = 1
      INTO var_IDplayer;
      CALL avvia_turno(var_IDplayer, OLD.`IDpartita`);
   END IF;
END
```

3. Questo trigger si occupa di verificare che dopo ogni update sull'attributo *'carri armati presenti'* di una tupla della tabella Territorio, il valore sia almeno pari ad uno, ovvero che sia rispettata la regola aziendale RA5.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Risiko_online`.`check_armate_territorio` BEFORE UPDATE ON `Territorio` FOR EACH ROW
BEGIN
```

```
-- controllo se numero di carri armati presenti vale almeno 1
   IF NEW.`carri armati presenti` < 1  THEN
SIGNAL SQLSTATE '45014' SET MESSAGE_TEXT = 'Su ogni territorio in tuo possesso deve essere presente almeno 1 carro armato.';
   END IF;
END
```

1. Unico trigger before insert, il suo compito è verificare che sia rispettata la regola aziendale RA8, ovvero che ad una partita possano accedere al massimo sei giocatori. Ciò viene assicurato sfruttando la funzione '*conta_pax(partita)'* che restituisce il numero di partecipanti alla partita passata come parametro.

```
CREATE DEFINER = CURRENT_USER TRIGGER `Risiko_online`.`Partecipa_verifica_max_players` BEFORE INSERT ON `Partecipa` FOR EACH ROW
BEGIN

   declare var_num_pax INT;
   SET var_num_pax = conta_pax(NEW.`partita`);

IF var_num_pax > 6 THEN
SIGNAL SQLSTATE '45017' SET MESSAGE_TEXT = 'La partita non accetta più giocatori, non è possibile parteciparvi.';
   END IF;
END
```

Eventi

Sono stati sviluppati due eventi che valgono per tutta l'istanza del database. Nel dettaglio si tratta dei due timer da due minuti per:

- avviare una partita eseguendo l'update del suo status e quindi attivando il codice del relativo trigger;
- passare il turno al giocatore successivo nei casi in cui un giocatore non compia alcuna azione entro i due minuti previsti dal timer.

```
SET GLOBAL event_scheduler = ON;
```

```
DELIMITER //
CREATE EVENT IF NOT EXISTS `Risiko_online`.`timeout_partita`
ON SCHEDULE EVERY 5 SECOND STARTS CURRENT_TIME ON COMPLETION PRESERVE
DO
BEGIN
UPDATE `Partita` SET `status_partita` = 'in_corso'
   WHERE current_time() > `timer_partita` + interval 2 minute
   AND `status_partita` = 'in_attesa';
END//

CREATE EVENT IF NOT EXISTS `Risiko_online`.`timeout_turno`
ON SCHEDULE EVERY 5 SECOND STARTS CURRENT_TIME ON COMPLETION PRESERVE
DO
BEGIN
        declare var_IDmatch INT;
   declare var_IDplayer VARCHAR(45);
   declare var_action_type ENUM('rinforzo', 'spostamento', 'attacco', 'no_azione');

        UPDATE `Turno` SET `tipo_azione` = 'no_azione'
   WHERE current_time() > `start_time` + interval 2 minute
   AND `tipo_azione` IS NULL;

SELECT `tipo_azione`,`giocatore`,`partita`
   FROM `Turno` ORDER BY `#turno` DESC LIMIT 1
   INTO var_action_type, var_IDplayer, var_IDmatch;

   IF var_action_type = 'no_azione' THEN
   CALL prossimo_turno(var_IDplayer, var_IDmatch);
   END IF;
END //
```

Viste

Non sono state implementate Viste.

Stored Procedures e transazioni

Si riportano di seguito le 25 stored procedure e funzioni elaborate per il database. Si annota che sono state inserite, come richiesto dalla consegna, delle tuple. In particolare per quanto riguarda la tabella Utente, sono stati inseriti i seguenti valori:

IDutente	password	tipo
mod_risiko	mod_risiko	Moderatore
giulio	Roma	Giocatore
agrippina	Colonia	Giocatore
adriano	Tivoli	Giocatore
alexander	Supertramp	Giocatore

- ***'registrazione_nuovo_giocatore'***: procedure che prevede la creazione e l'inserimento di una nuova utenza di tipo 'giocatore'. Viene gestito l'errore di 'duplicate key' per renderne la comprensione all'utente finale più immediata dato che non è detto che tutti i giocatori del Risiko conoscano i codici di errore di MySQL. L'insert su Utente viene eseguita con livello di isolamento READ UNCOMMITTED in quanto trattandosi di una operazione di scrittura che non comporta letture non sussiste la necessità di avere un livello di isolamento più alto e si preferisce garantire migliori prestazioni. Per quanto riguarda il campo *password*, alla stringa inserita dall'utente viene applicata la funzione di ashing SHA1 nonostante cominci ad essere considerata superata in termini di sicurezza.

```
CREATE PROCEDURE `registrazione_nuovo_giocatore` (IN var_IDplayer VARCHAR(45), IN var_password VARCHAR(45))
BEGIN
	declare exit handler for sqlexception
	begin
		rollback;
		resignal;
	 end;
   -- livello di isolamento standard read committed per evitare che più utenti tentino di registrarsi con lo stesso ID a causa di dirty reads
   -- si è pensato anche ad aumentare il livello di isolamento ma si è preferito garantire migliori performance considerata la situazione
	SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
    START TRANSACTION;
        INSERT INTO `Utente` (`IDutente`, `tipo`, `password`,`op_time`)
        VALUES(var_IDplayer, 'giocatore', SHA1(var_password), current_timestamp());
    COMMIT;
END
```

- ***'assegnazione_carri_armati'**:* Il seguente blocco di codice si occupa dell'assegnazione dei carri armati ad ogni giocatore una volta che questo termina un turno compiendo una azione. Per definire il numero di armate da assegnare si è fatto uso della funzione 'ceiling' per approssimare ad un numero intero. Viene quindi eseguito direttamente dal sistema e non richiede nessun input dal client.

```
CREATE PROCEDURE `assegnazione_carri_armati` (IN var_IDplayer VARCHAR(45))
BEGIN
declare var_IDmatch INT;
  declare var_carri_da_assegnare INT;

declare exit handler for sqlexception
  begin
rollback;
    resignal;
end;

  -- livello di isolamento standard
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SET var_IDmatch = recupera_partita(var_IDplayer);
    SET var_carri_da_assegnare = CEILING((conta_possedimenti(var_IDplayer, var_IDmatch))/3);
UPDATE `Utente` SET `armate_disponibili` = `armate_disponibili` + var_carri_da_assegnare
WHERE `IDutente` = var_IDplayer;

    -- poiche i carri armati si assegnano dopo un'azione consideriamo questa come una operazione
    -- e aggiorniamo il valore dell'attributo op_time
    CALL set_operation_time(var_IDplayer);
COMMIT;
END
```

- ***'set_partita':*** Tale procedure viene attivata dal trigger sulla tabella Partita dopo un update e si occupa di assegnare i turni ai giocatori in base al tempo di ingresso specificato nell'attributo *'t_entrata'* facendo uso di un cursore. In secundis definisce il numero di carri armati che deve ricevere ogni giocatore, crea e distribuisce in maniera randomica i 42 Territori del gioco. Come ultima attività esegue una update della tabella Utente per rendere disponibili ai giocatori solo i carri armati non necessari a risolvere il vincolo di avere almeno un carro armato posizionato su ogni territorio occupato.

```
CREATE PROCEDURE `set_partita` (IN var_IDmatch INT)
BEGIN
declare var_nome_ter VARCHAR(32);
   declare var_carri_disponibili INT;
   declare var_IDplayer VARCHAR(45);
   declare var_while_i INT;
   declare var_num_pax INT;
   declare var_n_turno INT;
declare done INT DEFAULT false;

   -- assegnazione del turno ai giocatori in base al tempo di ingresso come da specifica
   declare cur_pax CURSOR FOR
SELECT `giocatore`
      FROM `Partecipa`
      WHERE `partita` = var_IDmatch
      ORDER BY `t_entrata` ASC;
declare CONTINUE HANDLER FOR NOT FOUND SET done = true;
   SET var_n_turno = 1;
   OPEN cur_pax;
   assegna_turno: LOOP
FETCH cur_pax INTO var_IDplayer;
      IF done THEN
LEAVE assegna_turno;
END IF;
UPDATE `Partecipa` SET `num_turno` = var_n_turno
WHERE `giocatore` = var_IDplayer AND `partita` = var_IDmatch;
SET var_n_turno = var_n_turno + 1;
```

```
END LOOP;
   CLOSE cur_pax;

SET var_num_pax = conta_pax(var_IDmatch);

IF var_num_pax = 3 THEN SET var_carri_disponibili = 35;
ELSEIF var_num_pax = 4 THEN SET var_carri_disponibili = 30;
ELSEIF var_num_pax = 5 THEN SET var_carri_disponibili = 25;
ELSE SET var_carri_disponibili = 20;
END IF;

-- creazione e distribuzione randomica dei territori
SET var_while_i = 42;
WHILE var_while_i > 0 DO
-- seleziono casualmente un nome di territorio per cui l'istanza non è già stata creata
SELECT `nome` FROM `Nazione`
WHERE `nome` NOT IN (SELECT `nome` FROM `Territorio`
WHERE `Partita` = var_IDmatch)
ORDER BY RAND() LIMIT 1
INTO var_nome_ter;

-- assegnazione dei territori ai giocatori in partita
SELECT `giocatore` FROM `Partecipa`
WHERE `partita` = var_IDmatch
AND `num_turno` = (ABS(var_while_i-42) % var_num_pax) + 1
INTO var_IDplayer;
-- creo l'istanza e l'assegno al giocatore selezionato
INSERT INTO `Territorio`(`nome`, `Partita`, `occupante`)
VALUES (var_nome_ter, var_IDmatch, var_IDplayer);
-- di default viene impostato il numero di carri armati sul territorio ad 1
SET var_while_i = var_while_i - 1;
END WHILE;

-- update del numero di carri armati disponibili ai giocatori
   -- si sottrae al numero calcolato in precedenza, la quantità di carri armati
   -- necessari a garantire 1 armata su ogni territorio occupato
```

```
UPDATE `Utente` SET `armate_disponibili` = var_carri_disponibili -
conta_possedimenti(`IDutente`,var_IDmatch)
WHERE `IDutente` IN (SELECT `giocatore` FROM `Partecipa`
WHERE `partita` = var_IDmatch);
END
```

- ***'lista_stanze_disponibili'***: è una stored procedure che consente al giocatore di visualizzare quali sono le stanze in cui è possibile entrare e partecipare ad una Partita, tradotto in condizioni, si tratta di tutte le stanze con partite in stato di *attesa* e con meno di sei giocatori partecipanti. Ciò permette all'utente giocatore di verificare a quali stanze, e quindi a quali partite, può accedere, senza doverlo scoprire per tentativi. Come livello di isolamento si è definito un READ COMMITTED per evitare di restituire dati su cui stanno avvenendo operazioni di scrittura e che ad esempio hanno appena raggiunto il numero massimo di giocatori.

```
CREATE PROCEDURE `lista_stanze_disponibili` ()
BEGIN

declare exit handler for sqlexception
  begin
    rollback;
    resignal;
  end;
  -- impostiamo come read only in quanto ci interessa solo leggere
  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  SET TRANSACTION READ ONLY;
  START TRANSACTION;
            SELECT `IDstanza`,`idPartita`, COUNT(`giocatore`) as 'player'
    FROM `Stanza`
            JOIN `Partita` ON `Partita`.`stanza` = `Stanza`.`IDstanza`
    LEFT JOIN `Partecipa` ON `Partita`.`idPartita` = `Partecipa`.`partita`
    WHERE `status_partita` = 'in_attesa'
    GROUP BY `idPartita`
    HAVING COUNT(`giocatore`) < 6;
  COMMIT;
```

```
END
```

- ***'registrazione_nuovo_moderatore':*** Tale procedure permette ad un moderatore già registrato nel database di inserire le credenziali per un nuovo utente di tipo 'moderatore'.

```
CREATE PROCEDURE `registrazione_nuovo_moderatore` (IN var_username VARCHAR(45), IN var_password VARCHAR(45))
BEGIN
	-- abbassiamo il livello di isolamento per garantire più concorrenza (se mai fosse necessaria), si suppone che i moderatori sappiano chi inserire
	SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
	START TRANSACTION;
	INSERT INTO `Utente`(`IDutente`,`tipo`,`password`)
		VALUES(var_username,'moderatore',SHA1(var_password));
	COMMIT ;
END
```

- ***'set_operation_time':*** La seguente procedure esegue solo una operazione di aggiornamento sulla tabella Utente per aggiornare l'attributo che tiene traccia dell'istante più recente in cui il giocatore ha compiuto una operazione valida alle considerazioni del report del moderatore. Volendo tale update poteva essere integrata nelle varie procedure valide per l'aggiornamento dell'attributo in esame, per una questione di leggibilità del codice e di consistenza con gli specifici obiettivi delle varie stored procedures si è deciso di definirla a se.

```
CREATE PROCEDURE `set_operation_time` (IN var_IDplayer VARCHAR(45))
BEGIN
  UPDATE `Utente`
  SET `op_time` = current_timestamp()
  WHERE `Utente`.`IDutente` = var_IDplayer;
END
```

- ***'elimina_giocatore'*** procedura che si occupa di gestire i vari casi in cui un giocatore deve essere rimosso da una partita. Come livello di isolamento si è scelto REPEATABLE READ in

quanto vengono eseguite operazioni di lettura e non possiamo ammettere risultati differenti all'interno della transazione. Infine la delete dalla tabella Partecipa viene effettuata solo se la partita non è ancora iniziata per preservare il report del giocatore GR1.

```
CREATE PROCEDURE `elimina_giocatore` (IN var_IDleaver VARCHAR (45))
BEGIN
   declare var_IDmatch INT;
   declare var_ultimo_pax VARCHAR(45);
   declare var_num_pax INT;
   declare var_status_m ENUM('in_attesa','in_corso','terminata');

   declare exit handler for sqlexception
        begin
                rollback;
                resignal;
        end;

   SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
   START TRANSACTION;

     SET var_IDmatch = recupera_partita(var_IDleaver);
     SELECT `status_partita` FROM `Partita`
     WHERE `idPartita` = var_IDmatch
     INTO var_status_m;

     IF var_status_m <> 'terminata' THEN
                        -- caso in cui partita non ancora avviata ed in attesa di pax
                        IF var_status_m = 'in_attesa' THEN
                                -- delete della partecipazione del giocatore alla partita 'in_attesa'
                               DELETE FROM `Partecipa` WHERE `giocatore` = var_IDleaver AND
`partita` = var_IDmatch;
            -- se si hanno meno di 3 giocatori è necessario riavviare il timer della partita
            IF conta_pax(var_IDmatch) < 3 THEN
                                        UPDATE `Partita`
```

```
                    SET `timer_partita` = NULL
                    WHERE `Partita`.`idPartita` = var_IDmatch;
                END IF;
            END IF;
-- caso in cui la partita in_corso e il giocatore non è stato eliminato, ovvero ha ancora dei Territori: non cominciate se non intendete finire!
    IF (var_status_m = 'in_corso' AND conta_possedimenti(var_IDleaver, var_IDmatch) <> 0) THEN
                    SIGNAL SQLSTATE '45021' SET MESSAGE_TEXT = '\'Non cominciate se non intendete finire!\': non è possibile abbondare una partita in corso.';
-- Caso in cui invece il giocatore non sta provando ad uscire ma è stato eliminato dagli altri giocatori
    ELSE
        UPDATE `Partecipa`
        SET `eliminato` = 1
        WHERE `giocatore` = var_IDleaver AND `partita` = var_IDmatch;
    END IF;

    -- in tutti i casi si esegue la riassegnazione dei turni di gioco
    UPDATE `Partecipa` p_1
            JOIN (
                    SELECT `giocatore`, ROW_NUMBER() OVER (ORDER BY `num_turno` ASC) AS new_num_turno
                    FROM `Partecipa`
                    WHERE `partita` = var_IDmatch AND `giocatore` <> var_IDleaver) p_2
                    ON p_1.`giocatore` = p_2.`giocatore`
                    SET p_1.`num_turno` = p_2.new_num_turno
        WHERE p_1.`partita` = var_IDmatch AND p_1.`giocatore` <> var_IDleaver;
        END IF;
    COMMIT;
END
```

- ***'salta_turno'*** Questa procedure è stata pensata per permettere al giocatore di concludere subito un turno senza azioni, evitando cosi di attendere lo scadere del timer di due minuti e rendendo

l'esperienza di gioco più fluida. Ovviamente sono stati inseriti dei controlli per verificare che il giocatore la stia chiamando all'interno del suo turno di gioco e la partita sia 'in_corso'; a tal proposito si esegue una call ad un'altra procedure '*controlli_ogni_azione'* che analizzeremo in seguito.

```
CREATE PROCEDURE `salta_turno` (
IN var_IDplayer VARCHAR(45))

-- STORED PROCEDURE utile per passare subito al turno successivo evitando di attendere il timeout
BEGIN
	declare var_IDmatch INT;
   declare var_status_m ENUM('avviata','in_attesa','in_corso','terminata');
   declare var_num_turn INT;

   -- set delle variabili dichiarate: recuperiamo la partita ed il suo 'status'
	SET var_IDmatch = recupera_partita(var_IDplayer);
   SELECT `status_partita`
   FROM Partita
   WHERE `idPartita` = var_IDmatch
   INTO var_status_m;

   IF var_status_m <> 'in_corso' THEN
		IF var_status_m  = 'in_attesa' THEN
			SIGNAL SQLSTATE '45018' SET MESSAGE_TEXT = 'La partita non è ancora iniziata: Azione non disponibile';
		ELSEIF var_status_m = 'terminata' THEN
			SIGNAL SQLSTATE '45019' SET MESSAGE_TEXT = 'La partita è terminata.';
		END IF;
	END IF;

   -- CALL
	CALL controlli_ogni_azione(var_IDplayer, var_IDmatch);
```

```
-- aggiornamento dell'attributo tipo_azione in tabella Turno per anticipare il timeout
-- aggiornamento della tabella Turno, attributo tipo_azione per anticipare il timeout
    SELECT `#turno`
FROM `Turno` ORDER BY `#turno` DESC LIMIT 1
INTO var_num_turn;
UPDATE `Turno` SET `tipo_azione` = 'no_azione'
WHERE `#turno` =var_num_turn AND `tipo_azione` IS NULL;

END
```

- ***'spostamento_carri_armati':*** Il seguente blocco di codice contiene tutte le operazioni da svolgere ogni volta che un giocatore vuole eseguire l'azione di spostamento di carri armati tra due territori sotto il proprio dominio, tra cui: la verifica della liceità della richiesta di spostamento, ovvero se, è il turno del giocatore, i territori inseriti come parametro dal giocatore esistono, sono sotto il suo dominio e sono tra loro adiacenti. Si procede poi con i vari aggiornamenti sulle tabelle coinvolte per concludere con l'assegnazione di nuovi carri armati al giocatore che ha compiuto l'azione ed il passaggio al turno successivo. Dato che la verifica del numero di carri armati sul Territorio di 'partenza' delle truppe è eseguita anche da altre procedures, si è preferito implementarla una volta sola come trigger sulla tabella Territorio nei modi discussi al relativo paragrafo.

```
CREATE PROCEDURE `spostamento_carri_armati` (
IN var_IDplayer VARCHAR(45),
IN var_num_carri INT,
IN var_provenienza VARCHAR(32),
IN var_destinazione VARCHAR(32))
BEGIN
    declare var_IDmatch INT;
   declare var_check INT;
   declare var_num_effettivi INT;
   declare var_turno_giocatore INT;

   -- recupero la partita a cui sta partecipando var_IDplayer
    SET var_IDmatch = recupera_partita(var_IDplayer);
```

```
    CALL controlli_ogni_azione(var_IDplayer, var_IDmatch);
   CALL verifica_esistenza_nazione(var_provenienza);
    CALL verifica_esistenza_nazione(var_destinazione);

   -- verifica che lo spostamento sia lecito: territori sotto dominio di var_IDplayer
    SELECT COUNT(*) FROM `Territorio`
   WHERE Territorio.partita = var_IDmatch AND Territorio.occupante = var_IDplayer
   AND (`nome` = var_destinazione OR `nome` = var_provenienza)
   INTO var_check;
   IF var_check <> 2 THEN
            SIGNAL SQLSTATE '45006' SET MESSAGE_TEXT = 'Si possono spostare carri armati solo tra territori che siano entrambi di proprietà del giocatore!';
    END IF;

   -- verifica che i territori siano adiacenti
   SELECT COUNT(*) FROM `Adiacenza`
    WHERE (`nazione_1` = var_provenienza AND `nazione_2` = var_destinazione) OR (`nazione_1` = var_destinazione AND `nazione_2` = var_provenienza)
    INTO var_check;
    IF var_check <> 1 THEN
            SIGNAL SQLSTATE '45007' SET MESSAGE_TEXT = 'Si possono spostare carri armati solo tra territori adiacenti!';
    END IF;

   -- update dei due Territori, se non ci sono abbastanza carri armati il trigger solleverà il problema
    UPDATE Territorio SET `carri armati presenti` = `carri armati presenti`- var_num_carri
   WHERE Territorio.partita = var_IDmatch AND Territorio.nome = var_provenienza;

   UPDATE Territorio SET `carri armati presenti` = `carri armati presenti`+ var_num_carri
   WHERE Territorio.partita = var_IDmatch AND Territorio.nome = var_destinazione;

   -- Update tabella Turno: attributi tipo_azione
    SELECT max(`#turno`) FROM `Turno`
```

```
    WHERE `partita` = var_IDmatch AND `giocatore` = var_IDplayer
    INTO var_turno_giocatore;
    UPDATE `Turno` SET `tipo_azione` = 'spostamento'
    WHERE `#turno` = var_turno_giocatore;

    -- assegnamento i nuovi carri armati al giocatore e passiamo al prox turno
    CALL assegnazione_carri_armati(var_IDplayer);
     CALL prossimo_turno(var_IDplayer,var_IDmatch);
  END
```

- ***'crea_stanza':*** La seguente procedure realizza una nuova stanza di gioco e crea all'interno di questa una nuova partita. Queste due operazioni di insert vengono eseguite in maniera atomica e con livello di isolamento READ COMMITTED per evitare dirty reads causate da due o più moderatori che chiamano la procedure nello stesso istante.

```
CREATE PROCEDURE `crea_stanza` (var_IDroom INT)
BEGIN

  declare exit handler for sqlexception
  begin
      rollback;
      resignal;
   end;
      SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;
            INSERT INTO `Stanza`(`IDstanza`)
                  VALUES(var_IDroom);
            -- facciamo subito la insert di una nuova partita, di solito nelle stanze si entra per giocare
:)
            INSERT INTO `Partita`(`stanza`,`status_partita`)
                  VALUES(var_IDroom,'in_attesa');
      COMMIT;
END
```

- ***'report_moderatore':*** stored procedure per realizzare il report del moderatore definito nelle specifiche di progetto. Per garantire la consistenza del report le varie query dei dati sono inserite all'interno di una transazione con livello di isolamento REPETABLE READ.

```
CREATE PROCEDURE `report_moderatore` ()
BEGIN
declare exit handler for sqlexception
  begin
    rollback;
    resignal;
  end;
  -- serve un buon livello di isolamento per prevenire report non consistenti
  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
  SET TRANSACTION READ ONLY;

  START TRANSACTION;
            -- numero stanze con partite in corso
            SELECT COUNT(`IDstanza`) AS `stanze con partita in corso`
            FROM `Stanza` JOIN `Partita` ON `Partita`.`stanza` = `Stanza`.`IDstanza`
            WHERE `status_partita` = 'in_corso';
            -- numero totale di giocatori nelle partite 'in corso'
            SELECT `idPartita`, COUNT(*) AS `giocatori in partita`
            FROM `Partita` JOIN `Partecipa` ON `Partita`.`idPartita` = `Partecipa`.`partita`
            WHERE `status_partita` = 'in_corso'
            GROUP BY `idPartita`;
            -- numero totale di giocatori attivi nei precedenti 15' ma non in partita
            SELECT COUNT(*) AS `giocatori non in partita`
            FROM `Utente`
            WHERE `op_time` >= current_timestamp() - interval 15 minute
            AND `IDutente` NOT IN (SELECT `giocatore`
                                   FROM `Partecipa` JOIN `Partita` ON
`Partecipa`.`partita` = `Partita`.`idPartita`
                        WHERE `Partita`.`status_partita` <> 'terminata');
```

```
   COMMIT;
END
```

- ***'report_giocatore_GR1':*** Sempre per rispondere alle specifiche di progetto, con la seguente procedure si realizza il report per il giocatore contenente lo storico delle partite a cui quest'ultimo ha partecipato. Come nel caso del report del moderatore anche qui le informazioni vengono richieste atomicamente con una transazione con la differenza di poter abbassare il livello di isolamento a READ COMMITTED in quanto viene eseguita un'unica query composta da una join. Infine viene aggiornato, chiamando la relativa procedura, l'attributo che registra il timestamp dell'ultima azione del giocatore.

```
CREATE PROCEDURE `report_giocatore_GR1` (IN var_IDplayer VARCHAR(45))
BEGIN
        declare exit handler for sqlexception
   begin
        rollback;
        resignal;
    end;
   SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
   START TRANSACTION;
                SELECT `idPartita`,`stanza`,`winner`
                FROM `Partita` JOIN `Partecipa` ON `Partita`.`idPartita` = `Partecipa`.`partita`
                WHERE `Partita`.`status_partita` = 'terminata' AND `Partecipa`.`giocatore` =
var_IDplayer ;

                -- anche questa è una azione valida per il set di action_time
                CALL set_operation_time(var_IDplayer);
   COMMIT;
END
```

- ***'lancio_dei_dadi':*** procedura che simula il lancio dei dadi previsto dalle regole del Risiko qualora si svolga un attacco. Il valore di ogni dado è definito come l'approssimazione all'intero

successivo del prodotto tra la funzione *rand()* è il numero sei. Per quanto concerne il livello di isolamento si è scelto READ UNCOMMITTED visto che non sono previste all'interno della transazione operazioni di lettura suscettibili di anomalie.

```
CREATE PROCEDURE `lancio_dei_dadi` (
IN var_num_dadi_att INT,
IN var_num_dadi_dif INT,
OUT var_num_perso_att INT,
OUT var_num_perso_dif INT
)
BEGIN

   declare var_dado1_att INT DEFAULT NULL;
   declare var_dado2_att INT DEFAULT NULL;
   declare var_dado3_att INT DEFAULT NULL;
   declare var_dado1_dif INT DEFAULT NULL;
   declare var_dado2_dif INT DEFAULT NULL;
   declare var_dado3_dif INT DEFAULT NULL;
   declare var_dado_temp INT DEFAULT NULL;

   -- non ci sono problemi, posso abbassare il livello di isolamento
         SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
         START TRANSACTION;

         -- caso in cui attacco con 1 carro armato (1 dado)
         if var_num_dadi_att = 1 then
                  set var_dado1_att = ceiling(rand()*6);
   -- caso in cui attacco con 2 carri armati (2 dadi)
   elseif var_num_dadi_att = 2 then
                  set var_dado1_att = ceiling(rand()*6);
      set var_dado2_att = ceiling(rand()*6);
   -- caso in cui attacco con 3 carri armati (3 dadi)
         elseif var_num_dadi_att = 3 then
                  set var_dado1_att = ceiling(rand()*6);
```

```
    set var_dado2_att = ceiling(rand()*6);
    set var_dado3_att = ceiling(rand()*6);
end if;

        -- caso in cui difendo con 1 carro armato (1 dado)
        if var_num_dadi_dif = 1 then
                set var_dado1_dif = ceiling(rand()*6);
-- caso in cui difendo con 2 carri armati (2 dadi)
elseif var_num_dadi_dif = 2 then
                set var_dado1_dif = ceiling(rand()*6);
    set var_dado2_dif = ceiling(rand()*6);
-- caso in cui difendo con 3 carri armati (3 dadi)
        elseif var_num_dadi_dif = 3 then
                set var_dado1_dif = ceiling(rand()*6);
    set var_dado2_dif = ceiling(rand()*6);
    set var_dado3_dif = ceiling(rand()*6);
        end if;

-- riordinamento dei dadi attaccante
-- si può fare anche con logica più elegante ma matematica 'troppo complessa' per SQL)
if var_dado1_att > var_dado2_att then
                set var_dado_temp = var_dado1_att;
    set var_dado1_att = var_dado2_att;
    set var_dado2_att = var_dado_temp;
        end if;
         if var_dado2_att > var_dado3_att then
                set var_dado_temp = var_dado2_att;
    set var_dado2_att = var_dado3_att;
    set var_dado3_att = var_dado_temp;
        end if;
         if var_dado1_att > var_dado2_att then
                set var_dado_temp = var_dado1_att;
    set var_dado1_att = var_dado2_att;
    set var_dado2_att = var_dado_temp;
```

```
        end if;

-- riordinamento dei dadi difensore
if var_dado1_dif > var_dado2_dif then
                set var_dado_temp = var_dado1_dif;
    set var_dado1_dif = var_dado2_dif;
    set var_dado2_dif = var_dado_temp;
        end if;
         if var_dado2_dif > var_dado3_dif then
                set var_dado_temp = var_dado2_dif;
    set var_dado2_dif = var_dado3_dif;
    set var_dado3_dif = var_dado_temp;
        end if;
         if var_dado1_dif > var_dado2_dif then
                set var_dado_temp = var_dado1_dif;
    set var_dado1_dif = var_dado2_dif;
    set var_dado2_dif = var_dado_temp;
        end if;

-- confronto tra i vari dadi e scrittura delle var in output
set var_num_perso_att = 0;
        set var_num_perso_dif = 0;
if ((var_dado1_att is not NULL ) and (var_dado1_dif is not NULL)) then
                if var_dado1_att > var_dado1_dif then
                        set var_num_perso_dif = 1;
                else
                        set var_num_perso_att = 1;
                end if;
        end if;
if ((var_dado2_att is not NULL ) and (var_dado2_dif is not NULL)) then
                if var_dado2_att > var_dado2_dif then
                        set var_num_perso_dif = var_num_perso_dif + 1;
                else
                        set var_num_perso_att = var_num_perso_att + 1;
```

```
            end if;
        end if;
    if ((var_dado3_att is not NULL ) and (var_dado3_dif is not NULL)) then
            if var_dado3_att > var_dado3_dif then
                set var_num_perso_dif = var_num_perso_dif + 1;
            else
                set var_num_perso_att = var_num_perso_att + 1;
            end if;
        end if;
        COMMIT;
END
```

- ***'attacco':*** Pur avendo un nome conciso e auto esplicativo, la seguente procedura è una delle più lunghe e complesse. Analizziamola dividendola in tre sezioni:
    1° sezione: vengono eseguiti tutti i controlli necessari ad accertarsi che sia possibile procedere e che non siano stati passati parametri errati, quindi se gli Stati sono adiacenti, se i nomi inseriti corrispondono a delle Nazioni del gioco, ecc. e vengono previsti dei messaggi di errore per alcuni casi;
    2° sezione: è il 'kernel' della procedura, prevede il setting delle varie variabili utili alla procedure che simula il lancio dei dadi e la CALL della stessa. Tale parte si conclude con una serie di if case per aggiornare le varie tabelle in base al risultato dei dadi;
    3° sezione: qui si conclude l'attacco con l'aggiornamento della tabella Turno e le CALL per, rispettivamente, assegnare i nuovi carri armati in quanto il giocatore ha compiuto un'azione e per passare al prossimo turno. Si ricorda che, essendo comune a tutte e tre le azioni del gioco, l'aggiornamento del timestamp dell'ultima operazione eseguita dal giocatore viene effettuato dalla procedura *'assegnazione_carri_armati'*.

```
CREATE PROCEDURE `attacco` (
IN var_num_carri INT,
IN var_provenienza VARCHAR(32),
IN var_destinazione VARCHAR(32),
IN var_IDattaccante VARCHAR(45),
OUT var_esito INT,
OUT var_perdite_att INT
```

```
)
BEGIN
    declare var_IDmatch INT;
    declare var_IDroom INT;
    declare var_num_carri_presenti INT;
    declare var_n_dadi_att INT;
    declare var_n_dadi_dif INT;
    declare var_check INT DEFAULT 0;
    declare var_num_perso_att INT DEFAULT 0;
    declare var_num_perso_dif INT DEFAULT 0;
    declare var_IDdifensore VARCHAR(45);
    declare var_turno_giocatore INT;

    SET var_IDmatch = recupera_partita(var_IDattaccante);

    CALL verifica_esistenza_nazione(var_provenienza);
    CALL verifica_esistenza_nazione(var_destinazione);
    CALL controlli_ogni_azione(var_IDattaccante, var_IDmatch);
    SET var_esito = 0;

    -- verifica che il territorio da cui parte l'attacco sia dell'attaccante
    SELECT COUNT(*) FROM `Territorio`
    WHERE `nome` = var_provenienza AND `partita` = var_IDmatch
    AND `occupante` = var_IDattaccante
    INTO var_check;

    IF var_check <> 1 THEN
                SIGNAL SQLSTATE '45001' SET MESSAGE_TEXT = 'Il territorio da cui parte l\'attacco non è sotto il controllo dell\'attaccante.';
    END IF;

    -- verifica che i territori siano adiacenti
    SELECT COUNT(*) FROM `Adiacenza`
```

```
        WHERE (`nazione_1` = var_provenienza AND `nazione_2` = var_destinazione) OR (`nazione_1` = var_destinazione AND `nazione_2` = var_provenienza)
        INTO var_check;

        IF var_check <> 1 THEN
            SIGNAL SQLSTATE '45002' SET MESSAGE_TEXT = 'Non è possibile attaccare un territorio non adiacente a quello da cui parte l\'attacco';
        END IF;

    -- verifica che il territorio verso cui è diretto l'attacco non sia già dell'attaccante
    -- inoltre inserisce in var_IDdifensore l'eventuale giocatore occupante il Territorio
    SELECT `occupante`
    FROM `Territorio`
        WHERE `partita` = var_IDmatch AND `nome` = var_destinazione
        INTO var_IDdifensore;

        IF var_IDdifensore = var_IDattaccante THEN
            SIGNAL SQLSTATE '45003' SET MESSAGE_TEXT = 'Non è possibile attaccare territori che hai già conquistato, intendi forse fare uno spostamento?';
        END IF;

    -- verifica se numero carri armati dichiarati sia compreso tra 1 e 3 (più controllo da trigger su 1 armata di stanza ogni Territorio)
        IF var_num_carri < 1 or var_num_carri > 3 THEN
            SIGNAL SQLSTATE '45004' SET MESSAGE_TEXT = 'Il numero di carri armati deve essere compreso tra 1 e 3.';
        END IF;

    -- verifica se i carri armati richiesti sono presenti sul territorio di provenienza
        SELECT `carri armati presenti` FROM `Territorio`
    WHERE `partita` = var_IDmatch AND `occupante` = var_IDattaccante AND `nome` = var_provenienza
    INTO var_num_carri_presenti;
```

```
IF var_num_carri_presenti <= var_num_carri THEN
        SIGNAL SQLSTATE '45005' SET MESSAGE_TEXT = 'Non hai abbastanza carri armati per completare l\'attacco, diminuisci il numero di carri e ritenta.';
END IF;

    -- setting delle variabili relative al numero dei dadi e call alla procedura di lancio dei dadi
    SET var_n_dadi_att = var_num_carri;
SELECT `carri armati presenti`
FROM `Territorio`
WHERE `partita` = var_IDmatch AND `nome` = var_destinazione
INTO var_num_carri_presenti;
-- caso in cui il territorio del difensore ha almeno tanti carri quanti quelli dell'attaccante
IF var_num_carri_presenti >= var_num_carri THEN
        SET var_n_dadi_dif = var_n_dadi_att;
    ELSE -- caso in cui il difensore ha meno carri armati dell'attaccante
        SET var_n_dadi_dif = var_num_carri_presenti;
    END IF;

CALL lancio_dei_dadi(var_n_dadi_att,var_n_dadi_dif,var_num_perso_att,var_num_perso_dif);
SET var_perdite_att = var_num_perso_att;

-- caso in cui il difensore non vince tutti i lanci dei dadi
IF var_num_perso_dif <> 0 THEN
        -- UPDATE `Territorio` SET `carri armati presenti` = `carri armati presenti` - var_num_perso_dif
        -- WHERE `partita` = var_IDmatch AND `nome` = var_destinazione;
        SELECT `carri armati presenti`
    FROM `Territorio`
    WHERE `partita` = var_IDmatch AND `nome` = var_destinazione
    INTO var_check;
    SET var_check = var_check - var_num_perso_dif;
        -- sottocaso in cui il difensore perde il territorio: aggiornamento dell'occupante e dei carri armati presenti
        IF  var_check = 0 THEN
```

```
                    UPDATE `Territorio` SET `carri armati presenti` = var_num_carri -
var_num_perso_att, `occupante` = var_IDattaccante
                    WHERE `partita` = var_IDmatch AND `nome` = var_destinazione;

                    UPDATE `Territorio` SET `carri armati presenti` = `carri armati presenti` -
(var_num_carri - var_num_perso_att)
                    WHERE `partita` = var_IDmatch AND `nome` = var_provenienza;
                    SET var_esito = 1;
                        -- contiamo i rimanenti Territori del difensore per vedere subito se è
stato eliminato
                        -- se eliminato esegue il set del t_uscita e chiama la procedura
elimina_giocatore
          IF conta_possedimenti(var_IDdifensore, var_IDmatch) = 0 THEN
            call elimina_giocatore(var_IDdifensore);
                        END IF;

          -- verifica se l'attaccante ha conquistato tutti i Territori e quindi è il vincitore della partita
                        IF conta_possedimenti(var_IDattaccante, var_IDmatch) = 42 THEN
                            UPDATE `Partita` SET `status_partita` = 'terminata', `winner` =
var_IDattaccante
            WHERE `idPartita` = var_IDmatch;
                            -- avvia un'altra partita nella stessa stanza
            SELECT `stanza` FROM `Partita`
            WHERE `idPartita` = var_IDmatch
            INTO var_IDroom;
                            INSERT INTO `Partita`(`stanza`,`status_partita`)
                                VALUES(var_IDroom,'in_attesa');
                            -- set di var_esito a codice per identificare la vittoria per non
istanziare una nuova variable
            SET var_esito = 42;
                        END IF;
      END IF;
        END IF;
```

```
   -- caso in cui l'attaccante non vince tutti i lanci dei dadi
   IF var_num_perso_att <> 0 THEN
                UPDATE `Territorio` SET `carri armati presenti` = `carri armati presenti` -
var_num_perso_att
                WHERE `partita` = var_IDmatch AND `nome` = var_provenienza AND `occupante` =
var_IDattaccante;
   END IF;

   -- aggiornamento della table Turno
   SELECT max(`#turno`) FROM `Turno`
   WHERE `partita` = var_IDmatch AND `giocatore` = var_IDattaccante
   INTO var_turno_giocatore;

   -- update della tabella Turno: aggiorna attributo tipo_azione
   UPDATE `Turno` SET `tipo_azione` = 'attacco'
   WHERE `#turno` = var_turno_giocatore;

   -- assegniamo i nuovi carri armati al giocatore e avviamo il prox turno
   CALL assegnazione_carri_armati(var_IDattaccante);
   CALL prossimo_turno(var_IDattaccante, var_IDmatch);
END
```

- ***'displacement_carri_armati':*** si tratta dell'implementazione dell'azione di dispiegamento di nuovi carri armati su un Territorio occupato dal giocatore. Per quanto riguarda la verifica dell'effettiva disponibilità di armate da dispiegare sul Territorio scelto, si fa ricorso allo stesso schema procedura-trigger già visto in *'spostamento_carri_armati'*: la procedure effettua l'update della tabella su cui è presente un trigger che nel caso segnale l'errore.

```
CREATE PROCEDURE `displacement_carri_armati` (
IN var_IDplayer VARCHAR(45),
IN var_num_carri_disp INT,
IN var_destinazione VARCHAR(32))
BEGIN
   declare var_IDmatch INT;
```

```
declare var_turno_giocatore INT;
declare var_check VARCHAR(45);

SET var_IDmatch = recupera_partita(var_IDplayer);
CALL controlli_ogni_azione(var_IDplayer, var_IDmatch);
CALL verifica_esistenza_nazione(var_destinazione);

-- verifica che il giocatore stia attualmente occupando il territorio
SELECT `occupante`
FROM `Territorio`
WHERE `nome` = var_destinazione
INTO var_check;

IF var_IDplayer <> var_check THEN
		SIGNAL SQLSTATE '45020' SET MESSAGE_TEXT = 'Il territorio non è sotto il tuo dominio.';
END IF;
-- aggiornamento tabella Utente per il numero di armate disponibili,
-- in caso di non disponibilità il trigger dedicato si occuperà di sollevare l'errore
UPDATE `Utente` SET `armate_disponibili` = `armate_disponibili` - var_num_carri_disp
WHERE `IDutente` = var_IDplayer;

-- aggiornamento tabella Territorio
UPDATE `Territorio` SET `carri armati presenti` = `carri armati presenti` + var_num_carri_disp
WHERE `partita` = var_IDmatch AND `nome` = var_destinazione;

-- aggiornamento table Turno: tipo_azione
	SELECT max(`#turno`) FROM `Turno`
WHERE `partita` = var_IDmatch AND `giocatore` = var_IDplayer
INTO var_turno_giocatore;
UPDATE `Turno` SET `tipo_azione` = 'rinforzo'
WHERE `#turno` = var_turno_giocatore;

-- assegnamo i nuovi carri armati al giocatore e passiamo al prox turno
```

```
    CALL assegnazione_carri_armati(var_IDplayer);
    CALL prossimo_turno(var_IDplayer, var_IDmatch);
END
```

- ***'prossimo_turno':*** il codice seguente realizza il passaggio di turno tra i giocatori avviando il turno del giocatore con numero di turno successivo a quello del player passato come parametro di ingresso.

```
CREATE PROCEDURE `prossimo_turno` (IN var_IDplayer VARCHAR(45), IN var_IDmatch INT)
BEGIN
    declare var_action_type ENUM('attacco','spostamento','rinforzo','no_azione');
    declare var_num_turno INT;
    declare var_next_IDplayer VARCHAR(45);
    declare var_next_turn INT;
    declare var_num_pax SMALLINT;

    SET var_num_pax = conta_pax(var_IDmatch);

        SELECT `num_turno` FROM `Partecipa`
    WHERE `giocatore` = var_IDplayer AND `partita` = var_IDmatch
    INTO var_num_turno;

    -- definiamo il numero del turno successivo, ovvero var_next_turn
    SET var_next_turn = (var_num_turno % var_num_pax) + 1;

    -- grazie a var_next_turno possiamo ricavare var_next_IDplayer aka il giocatore successivo
    SELECT `giocatore`
    FROM `Partecipa`
    WHERE `num_turno`= var_next_turn AND `partita` = var_IDmatch
        INTO var_next_IDplayer;

    CALL avvia_turno(var_next_IDplayer,var_IDmatch);
END
```

- ***'avvia_turno':*** possiamo definirla come il sequel della stored procedure *'prossimo_turno'* in quanto viene sempre eseguita dopo di questa eccetto che nel caso in cui la partita sia appena avviata e si tratti del primo turno. Per tale motivo si è scelto di lasciarla come una procedure indipendente nonostante sia composta letteralmente da quattro righe.

```
CREATE PROCEDURE `avvia_turno` (IN var_IDplayer VARCHAR(45), in var_IDmatch INT)
BEGIN
    INSERT INTO `Turno`(`giocatore`,`partita`,`start_time`)
        VALUES (var_IDplayer, var_IDmatch, current_time());
END
```

- ***'controlli_ogni_azione':*** Stored procedure che raccoglie tutte le verifiche da eseguire ogni volta che un giocatore vuole compiere una delle tre azioni di gioco. Sono stati previsti dei messaggi di errore in base ai vari campi oggetto di verifica.

```
CREATE PROCEDURE `controlli_ogni_azione` (IN var_IDplayer VARCHAR(45), IN var_IDmatch INT)
BEGIN
    declare var_status_p ENUM('avviata', 'in_attesa', 'in_corso', 'terminata');
   declare var_IDplayer_check VARCHAR(45);
   declare var_elim_check SMALLINT;

    -- verifica se la partita sia nello status 'in_corso'
    SELECT `status_partita`
   FROM `Partita`
   WHERE `idPartita` = var_IDmatch
   INTO var_status_p;
   IF var_status_p = 'in_attesa' THEN
        SIGNAL SQLSTATE '45008' SET MESSAGE_TEXT = 'In attesa di altri giocatori, la partita si avvierà a breve.';
   END IF;
   IF var_status_p = 'terminata' THEN
        SIGNAL SQLSTATE '45009' SET MESSAGE_TEXT = 'La partita è terminata.';
   END IF;
```

```
    -- verifica che il giocatore possa giocare il turno e quindi compiere un'azione
    -- deve esserre verificato che: è il turno del giocatore e il giocatore non è stato eliminato
      SELECT `giocatore`
    FROM `Turno`
    WHERE `partita` = var_IDmatch
      ORDER BY `#turno` DESC LIMIT 1
    INTO var_IDplayer_check;
    IF var_IDplayer <> var_IDplayer_check THEN
            SIGNAL SQLSTATE '45010' SET MESSAGE_TEXT = 'Non è ancora il tuo turno di
gioco.';
    END IF;

    -- si è scelto di tenere le due condizioni separate senza join per avere discernere le due condizioni
    SELECT `eliminato`
    FROM `Partecipa`
    WHERE `partita` = var_IDmatch AND `giocatore` = var_IDplayer
    INTO var_elim_check;
    IF var_elim_check = 1 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Sei stato eliminato dalla
partita, puoi giocarne un\'altra o farti una passeggiata.';
    END IF;

END
```

- ***'verifica_esistenza_nazione':*** Procedura che permette di verificare l'esistenza del nome di un territorio all'interno della tabella Nazione. Viene quindi riferita da tutte le procedures che necessitano di verificare la correttezza del nome di un territorio.

```
CREATE PROCEDURE `verifica_esistenza_nazione` (IN var_nazione VARCHAR(32))
BEGIN
        IF var_nazione NOT IN (SELECT `nome` FROM `Nazione`) THEN
                    SIGNAL SQLSTATE '45012' SET MESSAGE_TEXT = 'Non esiste una
Nazione con il nome inserito.';
```

```
            END IF;
END
```

- ***'stato_avanzamento_partita':*** Questa procedure permette al giocatore di visualizzare lo stato del gioco in modo tale da conoscere prima di tentare un'azione, il numero di carri armati e gli occupanti dei vari Territori. La sua esecuzione non viene conteggiata come azione ed è possibile solo all'interno del proprio turno di gioco.

```
CREATE PROCEDURE `stato_avanzamento_partita` (var_IDplayer VARCHAR(45))
BEGIN
        declare var_IDmatch INT;
   declare exit handler for sqlexception
   begin
                rollback;
      resignal;
        end;
        SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
   SET TRANSACTION READ ONLY;
   START TRANSACTION;
                SET var_IDmatch = recupera_partita(var_IDplayer);
      CALL controlli_ogni_azione(var_IDplayer, var_IDmatch);

      -- dettagli stato avanzamento partita
      SELECT `IDutente` AS `username`,`armate_disponibili` AS `carri armati`, `num_turno` AS `turno`
      FROM `Utente` JOIN `Partecipa` ON `Partecipa`.`giocatore` = Utente.IDutente
      WHERE `partita` = var_IDmatch;

      SELECT `nome`,`occupante`,`carri armati presenti`
      FROM `Territorio`
      WHERE `Territorio`.`partita` = var_IDmatch;
   COMMIT;
END
```

- ***'aggiungi_giocatore_in_partita':*** Lo scopo primario della seguente procedura è la realizzazione della partecipazione di un giocatore ad una partita attraverso una insert sulla tabella Partecipa qualora sussistano le condizioni per farlo. Nel fare ciò registra il timestamp che sarà poi utilizzato dalla procedura che assegna i turni seguendo l'ordine di ingresso dei giocatori. Inoltre si occupa di contare, tramite apposita funzione, il numero di partecipanti alla partita per cui è stata invocata e nel caso siano pari almeno ad il numero minimo di tre, imposta il timer della Partita che verrà poi interrogato dal relativo evento per avviare la partita.
  La verifica del raggiungimento del massimo numero di partecipanti viene lasciata al trigger before insert sulla tabella Partecipa per evitare di appesantire la procedure.

```
CREATE PROCEDURE `aggiungi_giocatore_in_partita` (IN var_IDplayer VARCHAR(45), IN var_room INT, OUT var_IDmatch INT)
BEGIN
   declare var_status_match ENUM('in_attesa', 'in_corso', 'terminata');
   declare var_num_pax SMALLINT;
   declare var_check INT;
   declare var_IDmatch INT;

   declare exit handler for sqlexception
   begin
      rollback;
      resignal;
   end;

   -- Impostiamo come livello di isolamento READ COMITTED per prevenire ad esempio
   -- l'inserimento di un giocatore in una partita appena terminata  e altri casi simili
   SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
         START TRANSACTION;
                  -- verifica se il giocatore è già in un'altra partita
      SELECT COUNT(*)
                  FROM `Partecipa` JOIN `Partita` ON `Partecipa`.`partita` = `Partita`.`idPartita`
      WHERE `giocatore` = var_IDplayer AND ((`status_partita` = 'in_attesa') OR (`status_partita` = 'in_corso' AND `eliminato` = 0))
```

```
INTO var_check;

IF var_check <> 0 THEN
                SIGNAL SQLSTATE '45015' SET MESSAGE_TEXT = 'Il giocatore sta già partecipando ad una partita.';
        END IF;

-- troviamo la partita dalla stanza
SELECT `idPartita` FROM `Partita`
WHERE `status_partita` <> 'terminata'
AND `Partita`.`stanza` = var_room
INTO var_IDmatch;

-- verifica della consistenza status_partita rispetto all'operazione richiesta
SELECT `status_partita` FROM `Partita`
WHERE  `idPartita` = var_IDmatch
INTO var_status_match;

IF var_status_match <> 'in_attesa' THEN
                SIGNAL SQLSTATE '45016' SET MESSAGE_TEXT = 'Partita già in corso o terminata, non è possibile parteciparvi.';
END IF;

        -- insert del giocatore in partita segnando l'istante in cui ci entra
INSERT INTO `Partecipa`(`giocatore`,`partita`,`t_entrata`)
                VALUES (var_IDplayer,var_IDmatch,current_timestamp());

-- set del timer della partita se ci sono almeno tre giocatori
SET var_num_pax = conta_pax(var_IDmatch);
IF var_num_pax + 1 = 3 THEN
                UPDATE `Partita`
   SET `timer_partita` = current_time()
   WHERE `idPartita` = var_IDmatch;
        END IF;
```

```
   COMMIT;
END
```

- ***'login':*** Stored procedure per eseguire l'accesso al database, riceve in ingresso username e password per restituire come output la tipologia di utenza che ha eseguito il login.

```
CREATE PROCEDURE `login` (in var_username varchar(45), in var_password varchar(45), out var_tipo INT)
BEGIN
        declare var_user_tipo ENUM('giocatore', 'moderatore');

   select `tipo` from `Utente`
        where `IDutente` = var_username
        and `Password` = SHA1(var_password)
        into var_user_tipo;

                if var_user_tipo = 'moderatore' then
                        set var_tipo = 1;
                elseif var_user_tipo = 'giocatore' then
                        set var_tipo = 2;
                else
                        set var_tipo = 0;
                end if;
END
```

Funzioni

- ***'conta_possedimenti':*** funzione che, dati in input l'identificatore di un giocatore e quello della partita a cui sta partecipando, restituisce in una apposita variabile il numero di territori occupati dal giocatore durante la partita.

```
CREATE FUNCTION `conta_possedimenti` (var_IDplayer VARCHAR(45),var_IDmatch INT) RETURNS INT READS SQL DATA
BEGIN
```

```
        declare var_num_possedimenti INT;

    SELECT COUNT(*)
    FROM `Territorio`
    WHERE `occupante` = var_IDplayer AND `partita`= var_IDmatch
    INTO var_num_possedimenti;

    RETURN var_num_possedimenti;
END
```

- ***'conta_pax':*** tale funzione si occupa di contare il numero di giocatori che stanno partecipando alla partita il cui identificatore è passato come input.

```
CREATE FUNCTION `conta_pax` (var_IDmatch INT) RETURNS INT READS SQL DATA
BEGIN

   declare var_num_pax INT;

    SELECT COUNT(*)
    FROM `Partecipa`
    WHERE `Partecipa`.`partita` = var_IDmatch
    INTO var_num_pax;
    RETURN var_num_pax;
END
```

- ***'recupera_partita':*** grazie al seguente codice, dato l'id del giocatore, si recupera la partita più recente presente nella relazione Partecipa, ovvero la partita a cui il giocatore sta partecipando o l'ultima a cui ha preso parte.

```
CREATE FUNCTION `recupera_partita`(var_IDplayer VARCHAR(45)) RETURNS INT READS
SQL DATA
BEGIN
        declare var_IDmatch INT;

    SELECT `idPartita` FROM `Partita`
    JOIN `Partecipa` ON `Partecipa`.`partita` = `Partita`.`idPartita`
    WHERE `Partecipa`.`giocatore` = var_IDplayer
```

```
    ORDER BY `idPartita` DESC LIMIT 1
    INTO var_IDmatch;

        RETURN var_IDmatch;
END
```